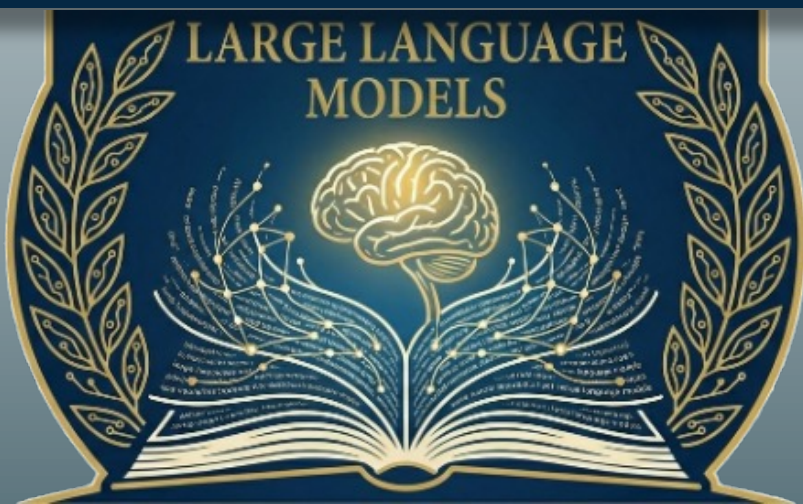


Basics



Mohammad Javad Dousti

Some slides are borrowed from CMU's Large Language Model Applications course.

Agenda

- ❑ What is a Language Model?
- ❑ Building Blocks of Language Models
- ❑ Decoding Strategies
- ❑ Transformers
- ❑ Perplexity
- ❑ Learning Objectives

What is a Language Model?

What is a Language Model?

- A language model is any model that outputs a probability distribution over the next token* in a sequence given the previous tokens in the sequence, that is: $P(y_t|y_{1:t-1})$.
- Historically, language models were statistical n-gram models. Instead of taking into account the full history of the sequence, they approximated this history by just looking back a few words.

*For now, let's assume token = word. We'll come back this.

What is a Language Model?

- ❑ **Example:** Suppose we are building a statistical language model using a text corpus, C.
 - We observe that the word “apple” follows the words “eat the” 2% of the times that “eat the” occurs in C.

- ❑ This means we’d set:

$$P(\text{“apple”} \mid \text{“eat the”}) = 0.02$$

- ❑ Since “eat the apple” is three words, we’d call this a 3-gram model.

Language models are not inherently generative.

Computing Sequence Likelihood

- Language models output the likelihood of the next word: $P(y_t|y_{1:t-1})$.
- Often, we will talk about the likelihood of an entire sequence $P(Y)=P(y_1, y_2, \dots, y_T)$.

Computing Sequence Likelihood

- Sequence likelihood can be computed from an LM using the chain rule:

$$P(["I", "eat", "the", "apple"]) =$$

$$P("apple" | ["I", "eat", "the"])*$$

$$P("the" | ["I", "eat"])*$$

$$P("eat" | ["I"])*P("I")$$

- In math:

$$P(Y)=P(y_1, y_2, \dots, y_T) = P(y_T|y_{1:T-1}) \times P(y_{T-1}|y_{1:T-2}) \times \dots \times P(y_1|\text{start of sequence})$$

Neural Language Models: Conditioned vs. Unconditioned

- Neural language models can either be
 - Designed to just predict the next word given the previous ones, or
 - Designed to predict the next word given the previous ones and some additional conditioning sequence.

Unconditioned: $P(Y)$

At each step the LM predicts:

$$P(y_t | y_{1:t-1})$$

Examples:

- GPT-2 / GPT-3
- LLaMA

Conditioned: $P(Y|X)$

At each step the LM predicts:

$$P(y_t | y_{1:t-1}, x_{1:T})$$

Examples:

- T5
- Most machine translation models

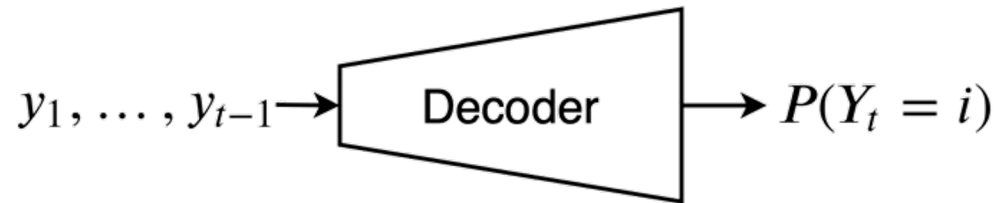
Sometimes called *sequence-to-sequence* or *seq2seq* models.

Building Blocks of Language Models

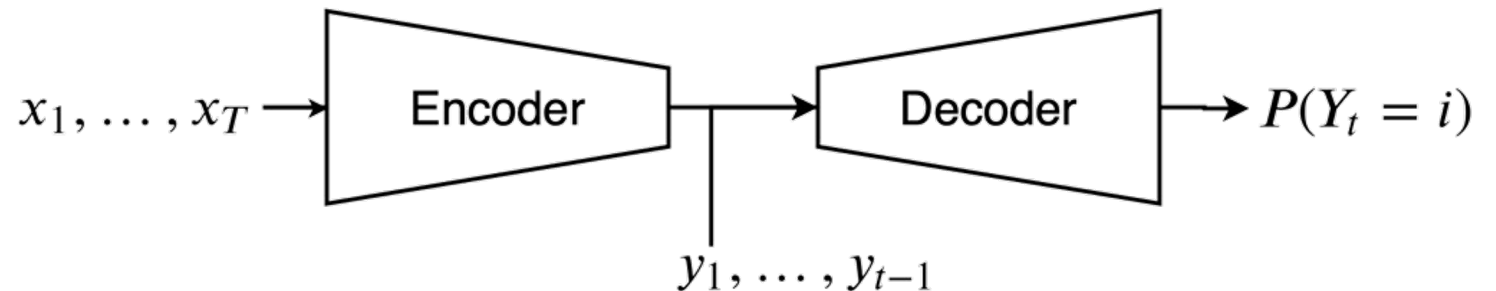
Neural Language Models: Conditioned vs. Unconditioned

- Unconditioned neural language models only have a decoder.
- Conditioned ones have an encoder and a decoder.

Unconditioned Language Model



Conditioned Language Model



There are also encoder-only models, but they aren't traditional language models.



Neural Language Models: Conditioned vs. Unconditioned

- ❑ Theoretically, any task designed for a decoder-only architecture can be turned into one for an encoder-decoder architecture, and vice-versa.

- ❑ **TASK: Continue the sequence.**

- ❑ Decoder-only version:

$P(Y=\text{“Once upon a time there lived a dreadful ogre.”})$

- ❑ Encoder-decoder version:

$P(Y=\text{“lived a dreadful ogre.”} \mid X=\text{“Once upon a time there”})$

Neural Language Models: Conditioned vs. Unconditioned

- ❑ Theoretically, any task designed for a decoder-only architecture can be turned into one for an encoder-decoder architecture, and vice-versa.

- ❑ **TASK: Translate from English to French.**

- ❑ Decoder-only version:

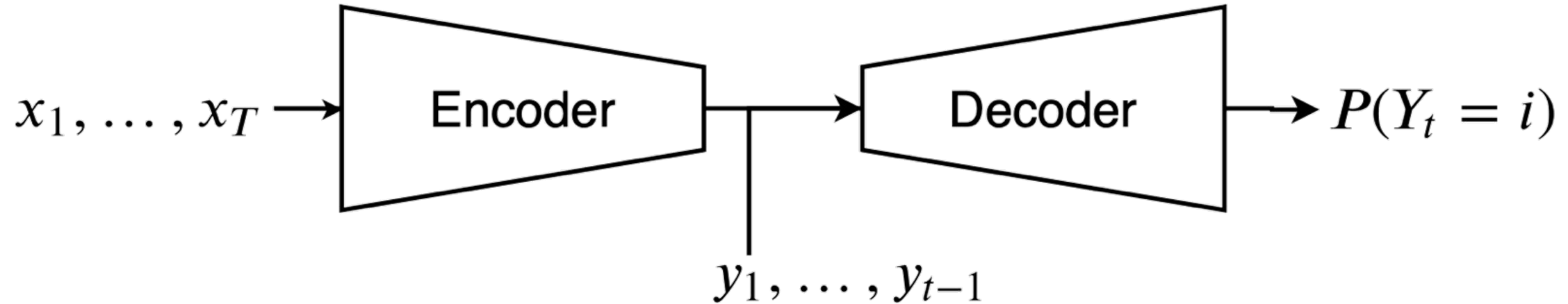
$P(Y=\text{"English: The hippo ate my homework. French: L'hippopotame a mangé mes devoirs."})$

- ❑ Encoder-decoder version:

$P(Y=\text{"L'hippopotame a mangé mes devoirs."} \mid X=\text{"The hippo ate my homework."})$

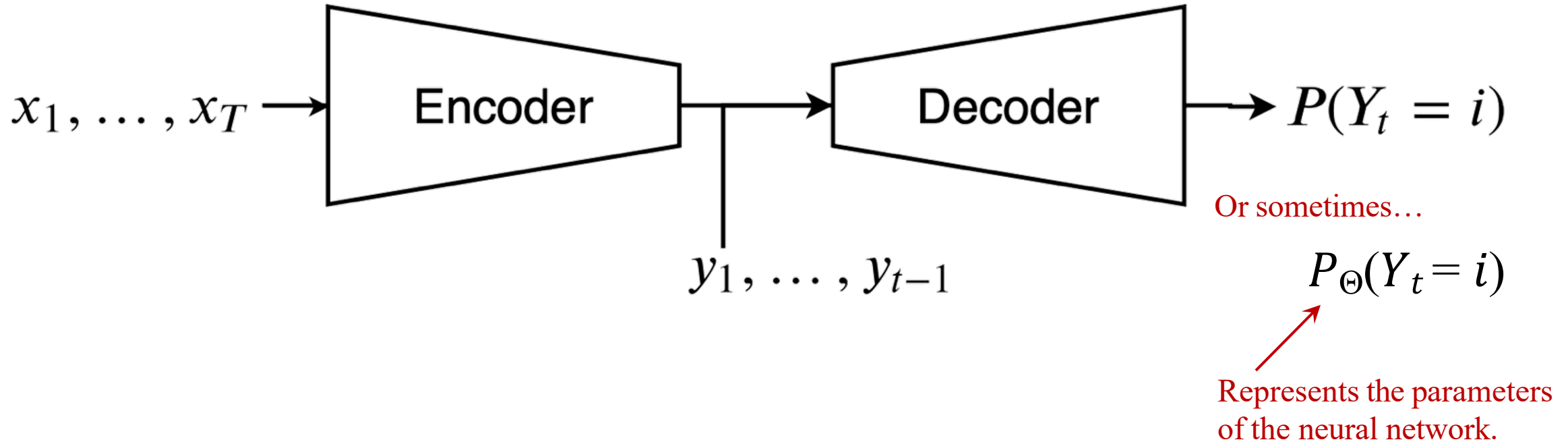
Summary of Terms You Should Know

- ❑ Input sequence: $x_1, \dots, x_T$
- ❑ Target sequence: $y_1, \dots, y_T$



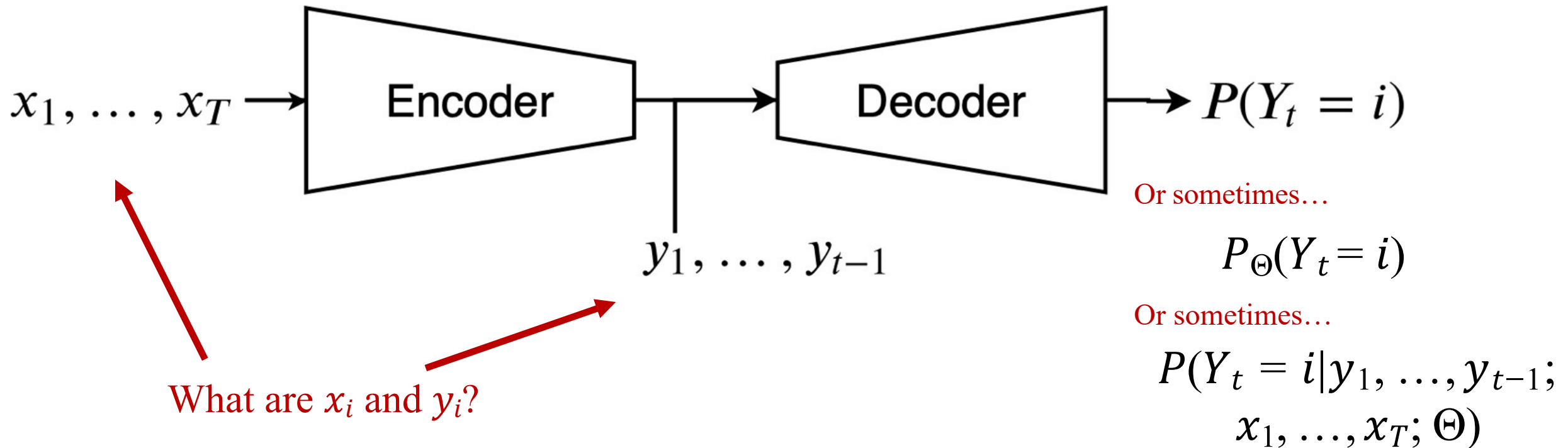
Summary of Terms You Should Know

- ❑ Input sequence: $x_1, \dots, x_T$
- ❑ Target sequence: $y_1, \dots, y_T$



Summary of Terms You Should Know

- Input sequence: $x_1, \dots, x_T$
- Target sequence: $y_1, \dots, y_T$



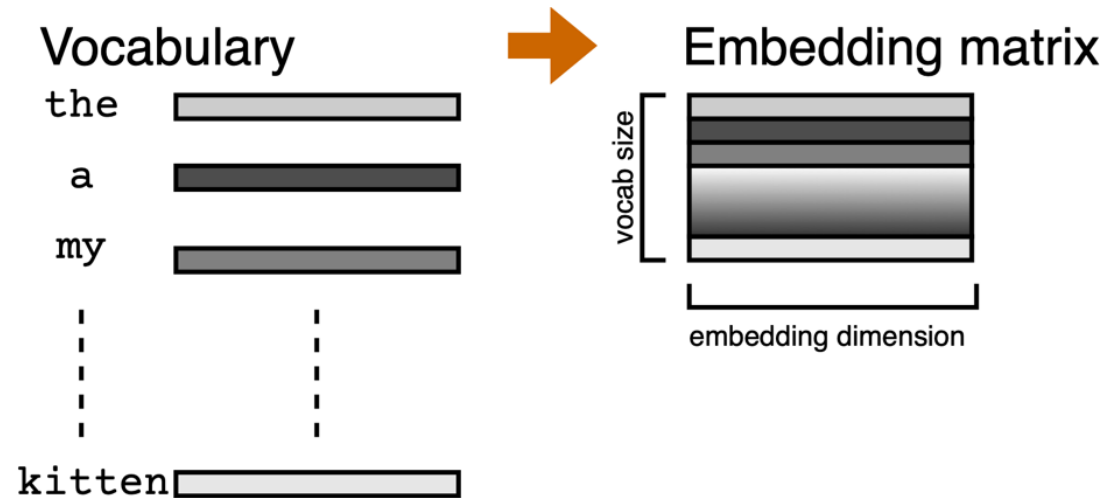
Tokenizing Text

- ❑ Tokenization is the task of taking text (or code or music) and turning it into a sequence of discrete items, called tokens.
- ❑ A *tokenizer* takes text and turns it into a sequence of discrete *tokens*.
- ❑ A *vocabulary* is the list of all available *tokens*.
- ❑ Let's tokenize: **A hippopotamus ate my homework.**

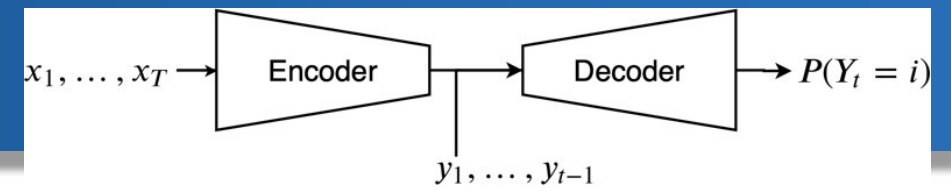
Vocab Type	Example	Length
Character level	['A', ' ', 'h', 'i', 'p', 'p', 'o', 'p', 'o', 't', 'a', 'm', 'u', 's', ' ', 'a', 't', 'e', ' ', 'm', 'y', ' ', 'h', 'o', 'm', 'e', 'w', 'o', 'r', 'k', '.']	31
Subword level	['A', 'hip', '##pop', '##ota', '##mus', 'ate', 'my', 'homework', '.']	9
Word level	['A', 'hippopotamus', 'ate', 'my', 'homework']	5

Turning Discrete Tokens into Continuous Vectors

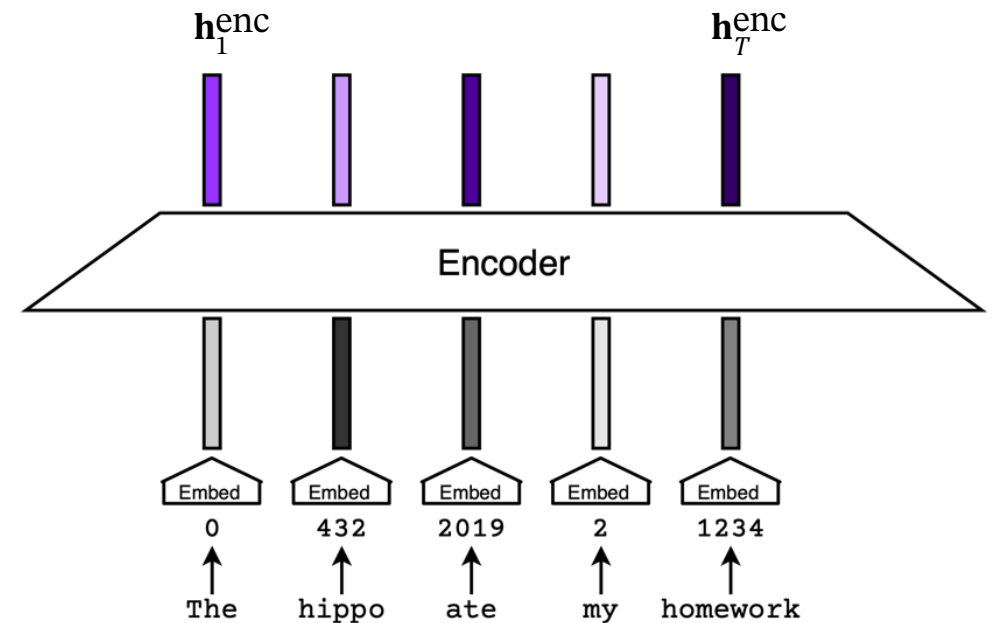
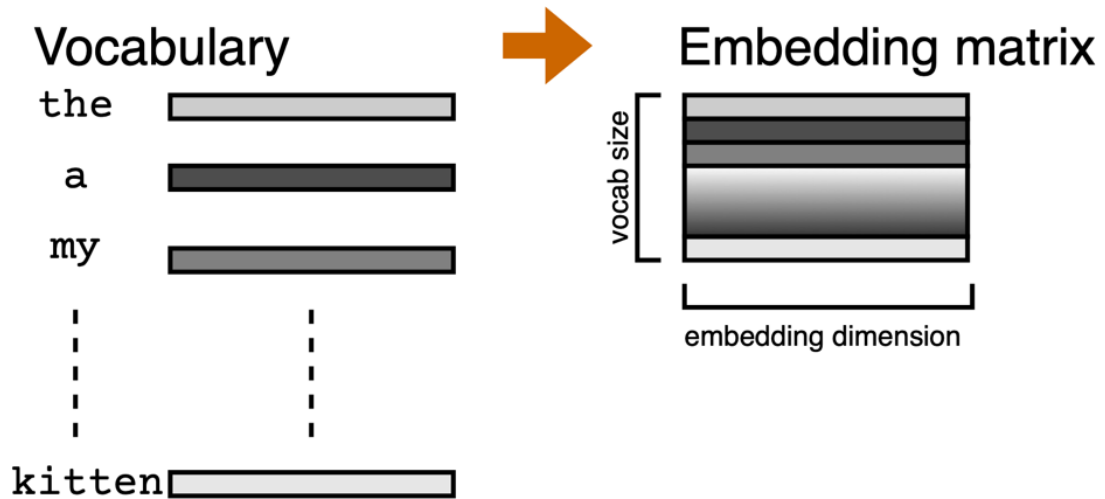
- ❑ Neural networks cannot operate on discrete tokens.
- ❑ Instead, we build an embedding matrix which associates each token in the vocabulary with a vector embedding.



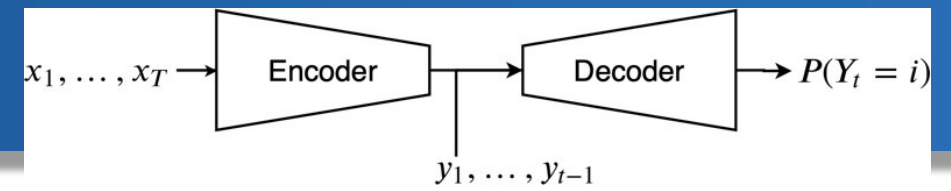
Encoder Inputs and Outputs



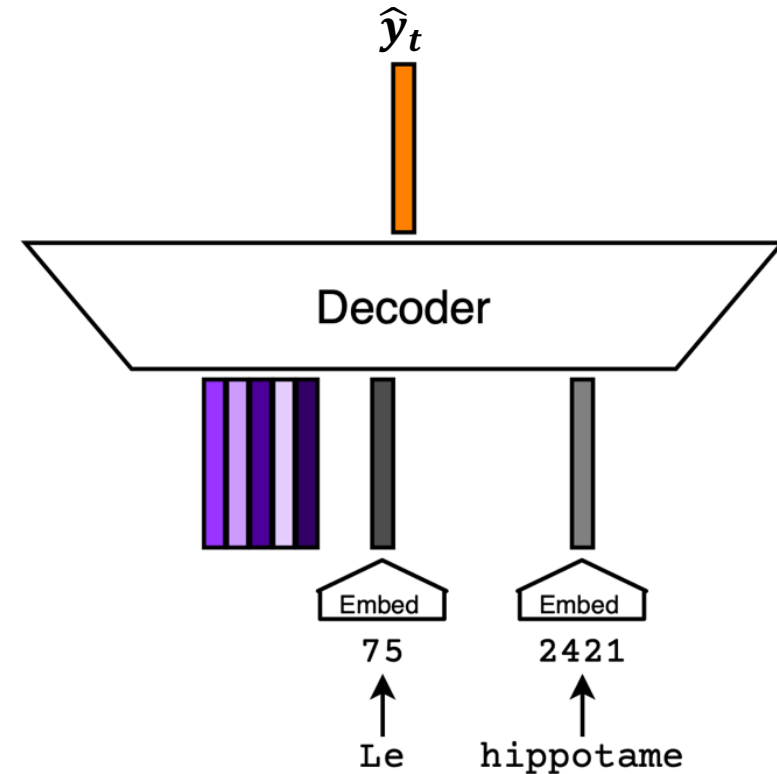
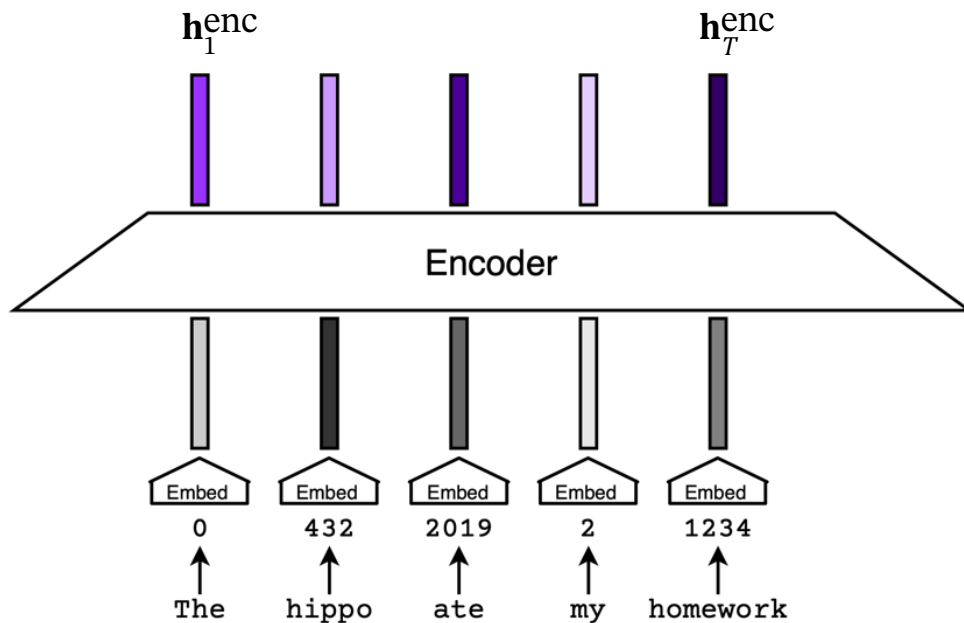
- ❑ The encoder takes as input the vector representations of each token in the input sequence.
- ❑ The encoder outputs a sequence of embeddings called *hidden states*.



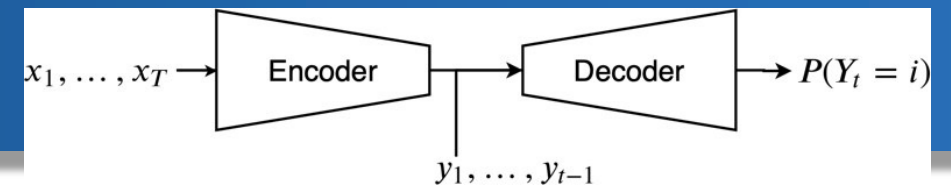
Decoder Inputs and Outputs



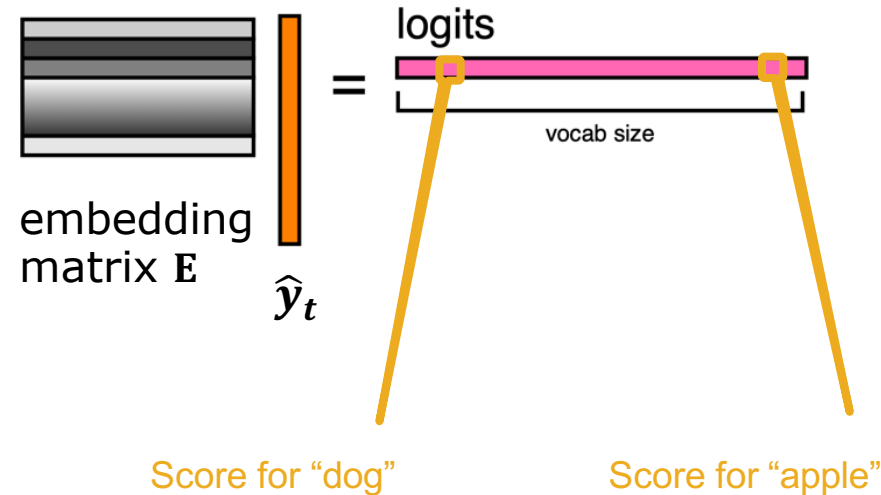
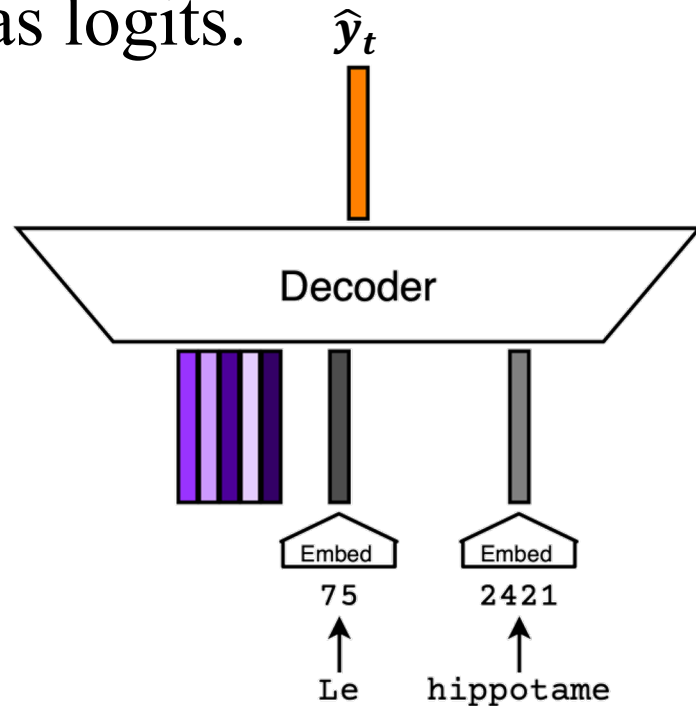
- ❑ The decoder takes as input the hidden states from the encoder as well as the embeddings for the tokens seen so far in the target sequence.
- ❑ It outputs an embedding $\hat{y}_t$.



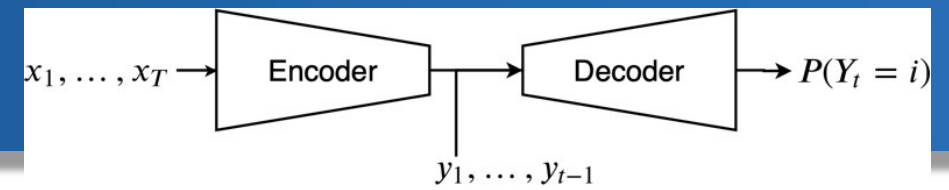
Decoder Inputs and Outputs



- ❑ Ideally, $\hat{\mathbf{y}}_t$ would be as close as possible to the embedding of the true next token.
- ❑ We multiply the predicted embedding, $\hat{\mathbf{y}}_t$ by our vocabulary embedding matrix $\mathbf{E}$ to get a score for each vocabulary word. These scores are referred to as logits.



Decoder Inputs and Outputs



- We multiply the predicted embedding $\hat{\mathbf{y}}_t$ by our vocabulary embedding matrix to get a score for each vocabulary word. These scores are referred to as logits.
- The *softmax function* is used to turn the logits into probabilities.

$$P(Y_t = i | \mathbf{x}_{1:T}, \mathbf{y}_{1:t-1}) = \frac{\exp(\mathbf{E}\hat{\mathbf{y}}_t[i])}{\sum_j \exp(\mathbf{E}\hat{\mathbf{y}}_t[j])}$$

- Example: Suppose we are trying to predict the 5th word in the sequence “*the dog chased the*”. We want to know the probability the next word is “*cat*”.

$$P(Y_5 = \text{"cat"} | \text{"the dog chase the"}) = \frac{\exp(\text{score in logits for "cat"})}{\text{normalization term}} = 0.321$$

Loss Function: Negative Log Likelihood

$$\mathcal{L} = - \sum_{t=1}^T \log P(Y_t = i^* | \mathbf{x}_{1:T}, \mathbf{y}_{1:t-1})$$

The index of the true t^{th} word in the target sequence.

The probability the language model assigns to the true t^{th} word in the target sequence.

Loss Function: Negative Log Likelihood

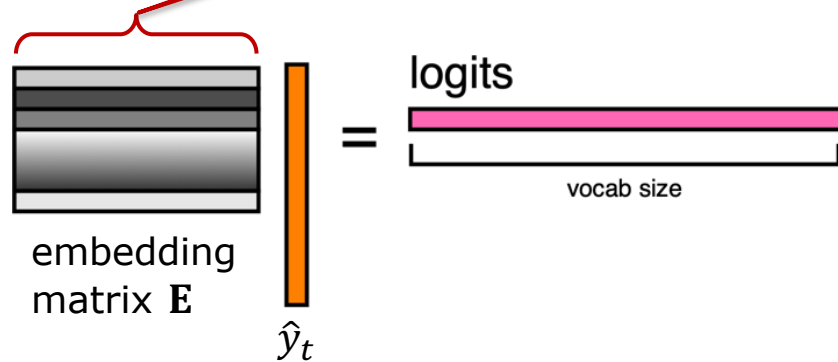
$$\begin{aligned}\mathcal{L} &= - \sum_{t=1}^T \log P(Y_t = i^* | \mathbf{x}_{1:T}, \mathbf{y}_{1:t-1}) \\ &= - \sum_{t=1}^T \log \frac{\exp(\mathbf{E} \hat{\mathbf{y}}_t[i^*])}{\sum_j \exp(\mathbf{E} \hat{\mathbf{y}}_t[j])}\end{aligned}$$

Recall:

$$P(Y_t = i | X_{1:T}, y_{1:t-1}) = \frac{\exp(\hat{y}_t[i])}{\sum_j \exp(\hat{y}_t[j])}$$

Loss Function: Negative Log Likelihood

$$\begin{aligned}\mathcal{L} &= - \sum_{t=1}^T \log P(Y_t = i^* | \mathbf{x}_{1:T}, \mathbf{y}_{1:t-1}) \\ &= - \sum_{t=1}^T \log \frac{\exp(\mathbf{E} \hat{\mathbf{y}}_t[i^*])}{\sum_j \exp(\mathbf{E} \hat{\mathbf{y}}_t[j])}\end{aligned}$$

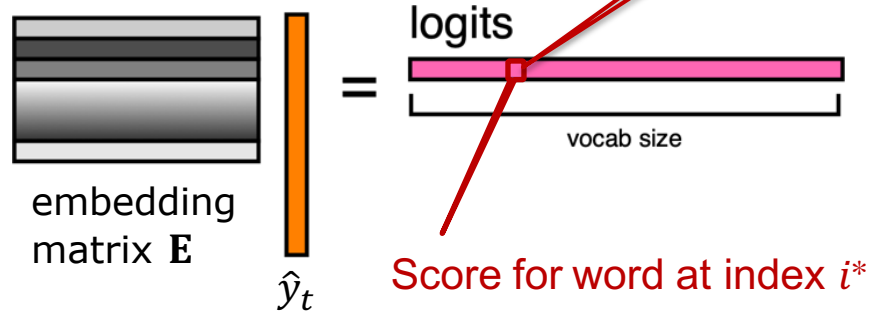


Recall:

$$P(Y_t = i | X_{1:T}, y_{1:t-1}) = \frac{\exp(\hat{\mathbf{y}}_t[i])}{\sum_j \exp(\hat{\mathbf{y}}_t[j])}$$

Loss Function: Negative Log Likelihood

$$\begin{aligned}\mathcal{L} &= - \sum_{t=1}^T \log P(Y_t = i^* | \mathbf{x}_{1:T}, \mathbf{y}_{1:t-1}) \\ &= - \sum_{t=1}^T \log \frac{\exp(\mathbf{E} \hat{\mathbf{y}}_t[i^*])}{\sum_j \exp(\mathbf{E} \hat{\mathbf{y}}_t[j])}\end{aligned}$$



Recall:

$$P(Y_t = i | X_{1:T}, y_{1:t-1}) = \frac{\exp(\hat{\mathbf{y}}_t[i])}{\sum_j \exp(\hat{\mathbf{y}}_t[j])}$$

Loss Function: Negative Log Likelihood

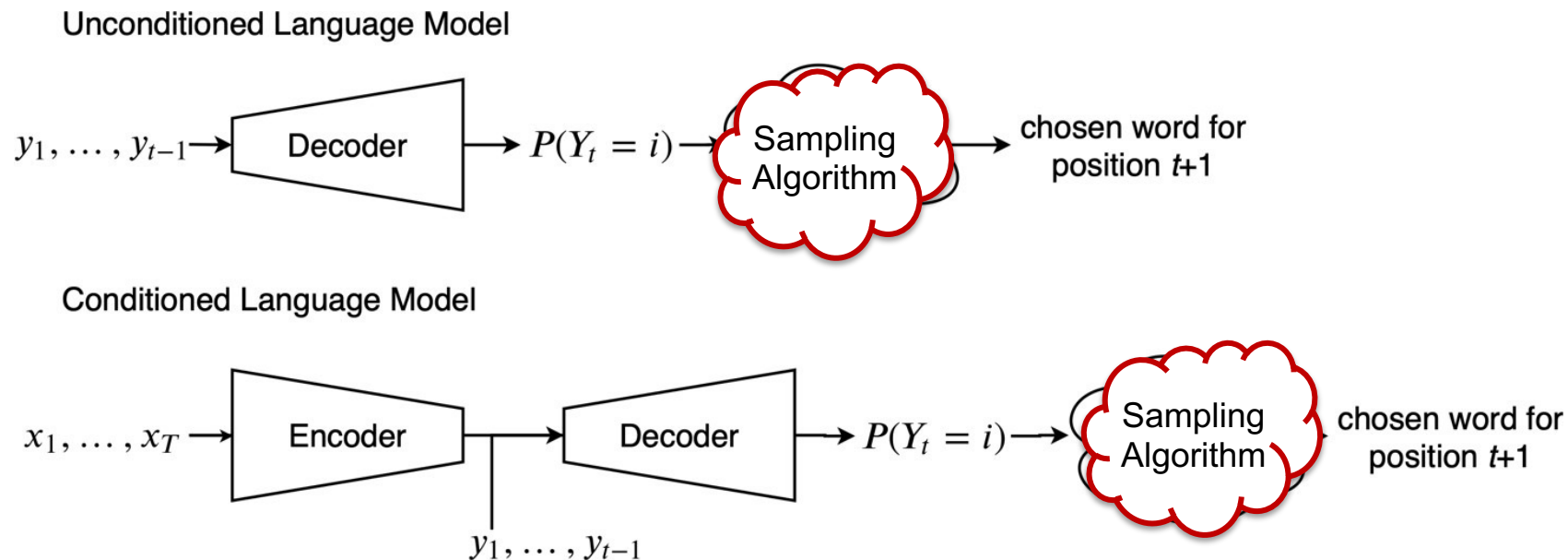
$$\begin{aligned}\mathcal{L} &= - \sum_{t=1}^T \log P(Y_t = i^* | \mathbf{x}_{1:T}, \mathbf{y}_{1:t-1}) \\ &= - \sum_{t=1}^T \log \frac{\exp(\mathbf{E} \hat{\mathbf{y}}_t[i^*])}{\sum_j \exp(\mathbf{E} \hat{\mathbf{y}}_t[j])} \\ &= - \sum_{t=1}^T \mathbf{E} \hat{\mathbf{y}}_t[i^*] - \log \sum_j \exp(\mathbf{E} \hat{\mathbf{y}}_t[j])\end{aligned}$$

Recall

$$P(Y_t = i | X_{1:T}, y_{1:t-1}) = \frac{\exp(\hat{y}_t[i])}{\sum_j \exp(\hat{y}_t[j])}$$

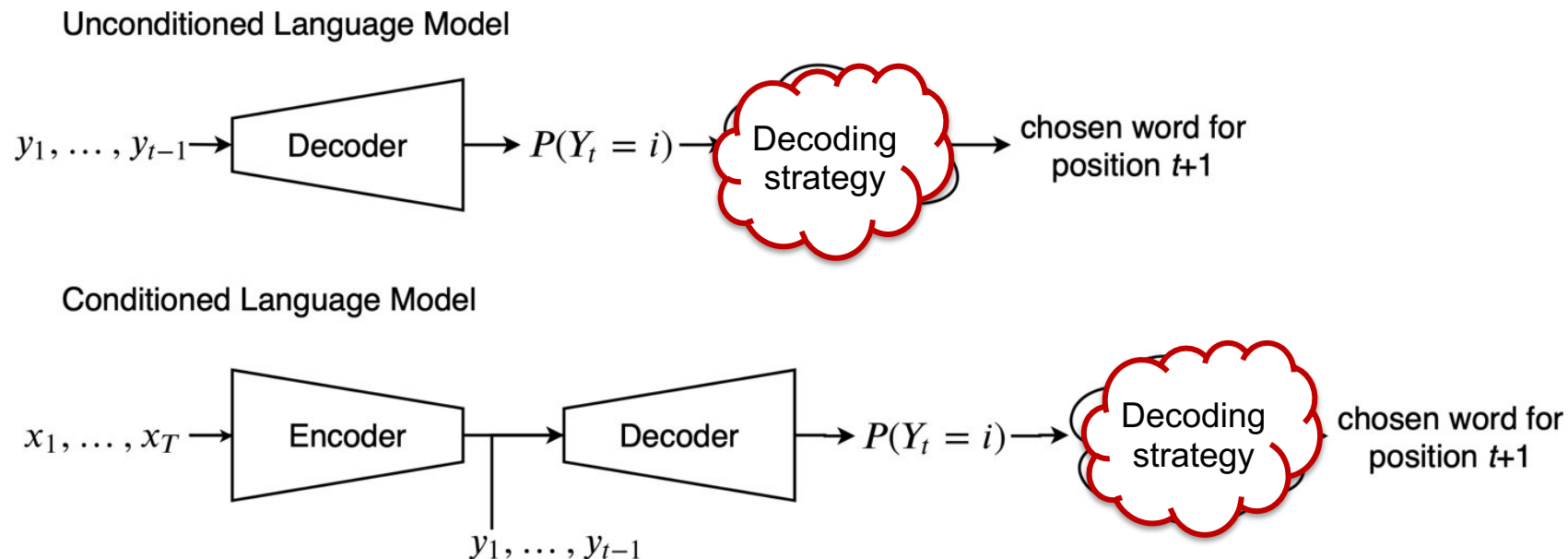
Now how do we do generation?

- To do generation, we need a **sampling algorithm** that selects a word given the predicted probability distribution $P(Y_t = i | y_{1:t-1})$.



Now how do we do generation?

- To do generation, we need a **sampling algorithm** that selects a word given the predicted probability distribution $P(Y_t = i | y_{1:t-1})$.



Decoding Strategies

How can we sample from
 $P(Y_t = i | y_{1:t-1})$?

How can we sample from $P(Y_t = i | y_{1:t-1})$?

- **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i | y_{1:t-1})$

Suppose our vocab consists of 4 words:

$$\mathcal{V} = \{apple, banana, orange, plum\}$$

We have prompted our LM with “apple apple” and want to generate the next word in the sequence.

Our language model predicts:

$$P(Y_3 = apple \mid Y_1 = apple, Y_2 = apple) = 0.05$$

$$P(Y_3 = banana \mid Y_1 = apple, Y_2 = apple) = 0.65$$

$$P(Y_3 = orange \mid Y_1 = apple, Y_2 = apple) = 0.2$$

$$P(Y_3 = plum \mid Y_1 = apple, Y_2 = apple) = 0.1$$

If we sample with argmax, what word would get selected?

(a) apple (b) banana (c) orange (d) plum

How can we sample from $P(Y_t = i | y_{1:t-1})$?

- ❑ **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i | y_{1:t-1})$
- ❑ **Option 2:** Randomly sample from the distribution returned by the model.

Suppose our vocab consists of 4 words:

$\mathcal{V} = \{apple, banana, orange, plum\}$

We have prompted our LM with “apple apple” and want to generate the next word in the sequence.

Our language model predicts:

$P(Y_3 = apple | Y_1 = apple, Y_2 = apple) = 0.05$

$P(Y_3 = banana | Y_1 = apple, Y_2 = apple) = 0.65$

$P(Y_3 = orange | Y_1 = apple, Y_2 = apple) = 0.2$

$P(Y_3 = plum | Y_1 = apple, Y_2 = apple) = 0.1$

With random sampling, what is the probability we'll pick “banana”?

(a) 0% (b) 5% (c) 65% (d) 100%

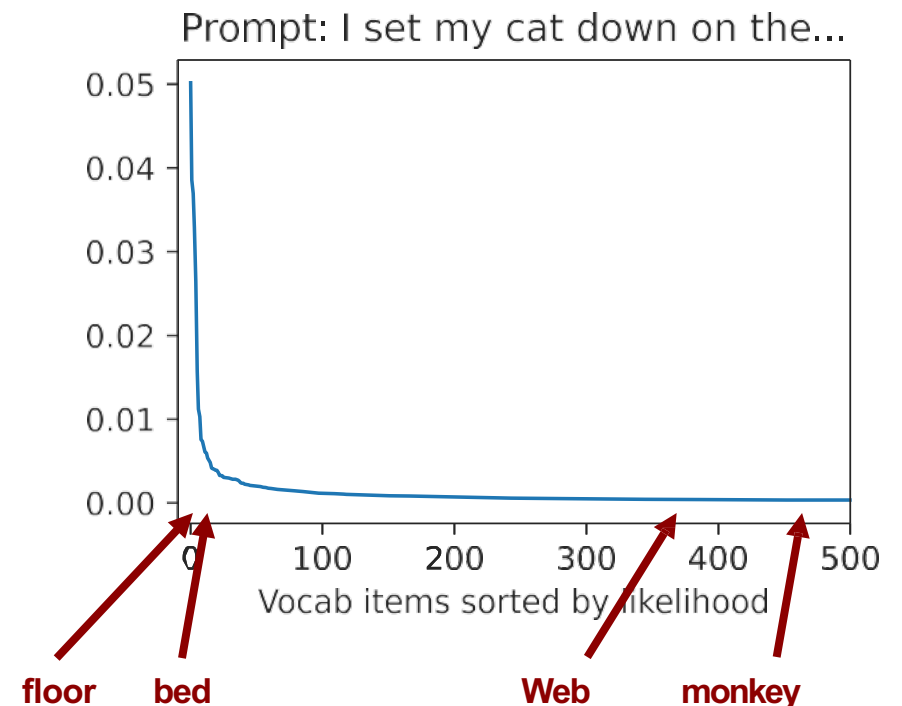
How can we sample from $P(Y_t = i | y_{1:t-1})$?

- ❑ **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i | y_{1:t-1})$
- ❑ **Option 2:** Randomly sample from the distribution returned by the model.

Problem with Random Sampling

Most tokens in the vocabulary get assigned very low probabilities but cumulatively, choosing any one of these low-probability tokens is pretty likely. In the example on the right, there is over a 29% chance of choosing a token v with $P(Y_t = v) \leq 0.01$.

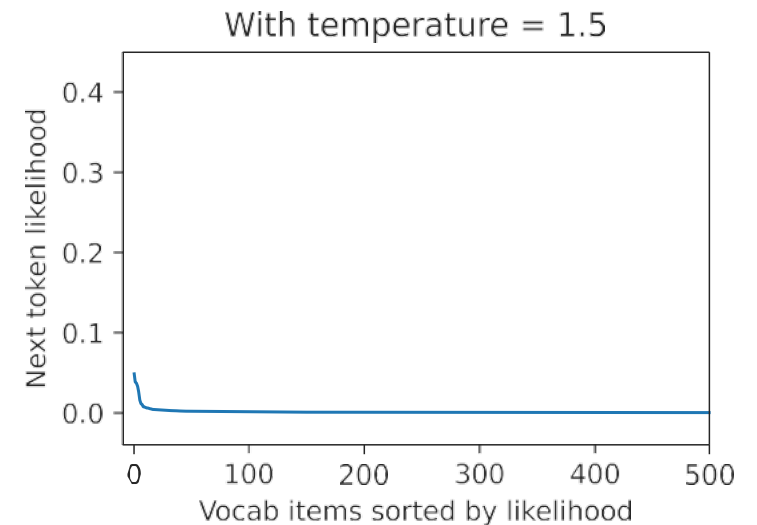
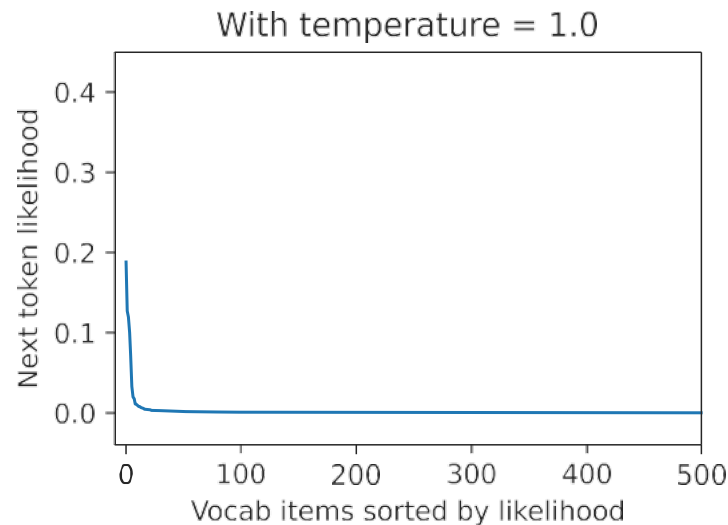
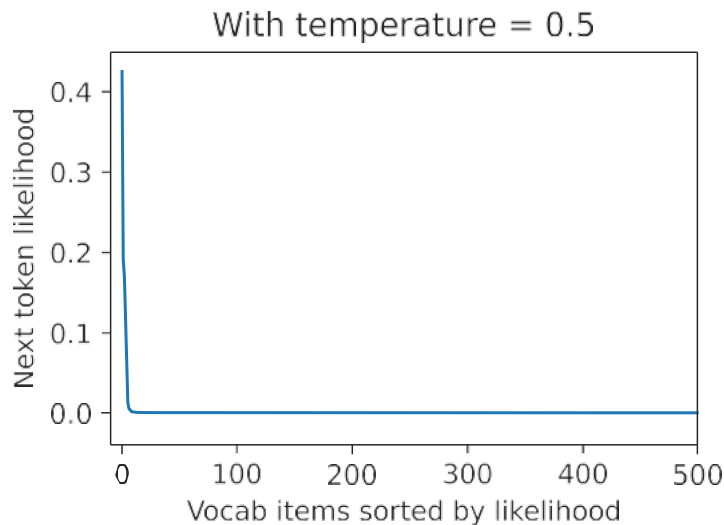
Solution: Modify the distribution returned by the model to make the tokens in the tail less likely.



How can we sample from $P(Y_t = i|y_{1:t-1})$?

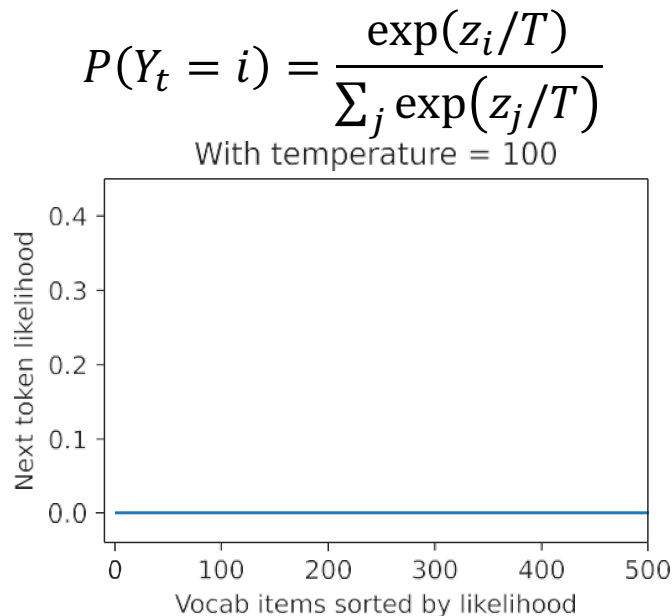
- ❑ **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i|y_{1:t-1})$
- ❑ **Option 2:** Randomly sample from the distribution returned by the model.
- ❑ **Option 3:** Randomly sample with temperature.

$$P(Y_t = i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$



How can we sample from $P(Y_t = i | y_{1:t-1})$?

- ❑ **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i | y_{1:t-1})$
- ❑ **Option 2:** Randomly sample from the distribution returned by the model.
- ❑ **Option 3:** Randomly sample with temperature.



Suppose our vocab consists of 4 words:

$\mathcal{V} = \{\text{apple}, \text{banana}, \text{orange}, \text{plum}\}$

We have prompted our LM with “apple apple” and want to generate the next word in the sequence.

Our language model predicts:

$P(Y_3 = \text{apple} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.05$

$P(Y_3 = \text{banana} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.65$

$P(Y_3 = \text{orange} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.2$

$P(Y_3 = \text{plum} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.1$

What would the probability of selecting “banana” be if we use temperature sampling and set $T = \infty$?

(a) 0% (b) 25% (c) 65% (d) 100%

How can we sample from $P(Y_t = i | y_{1:t-1})$?

- ❑ **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i | y_{1:t-1})$
- ❑ **Option 2:** Randomly sample from the distribution returned by the model.
- ❑ **Option 3:** Randomly sample with temperature.

$$P(Y_t = i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

As T approaches 0, random sampling with temperature looks more and more like argmax .

Suppose our vocab consists of 4 words:

$\mathcal{V} = \{\text{apple}, \text{banana}, \text{orange}, \text{plum}\}$

We have prompted our LM with “apple apple” and want to generate the next word in the sequence.

Our language model predicts:

$P(Y_3 = \text{apple} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.05$
 $P(Y_3 = \text{banana} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.65$
 $P(Y_3 = \text{orange} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.2$
 $P(Y_3 = \text{plum} \mid Y_1 = \text{apple}, Y_2 = \text{apple}) = 0.1$

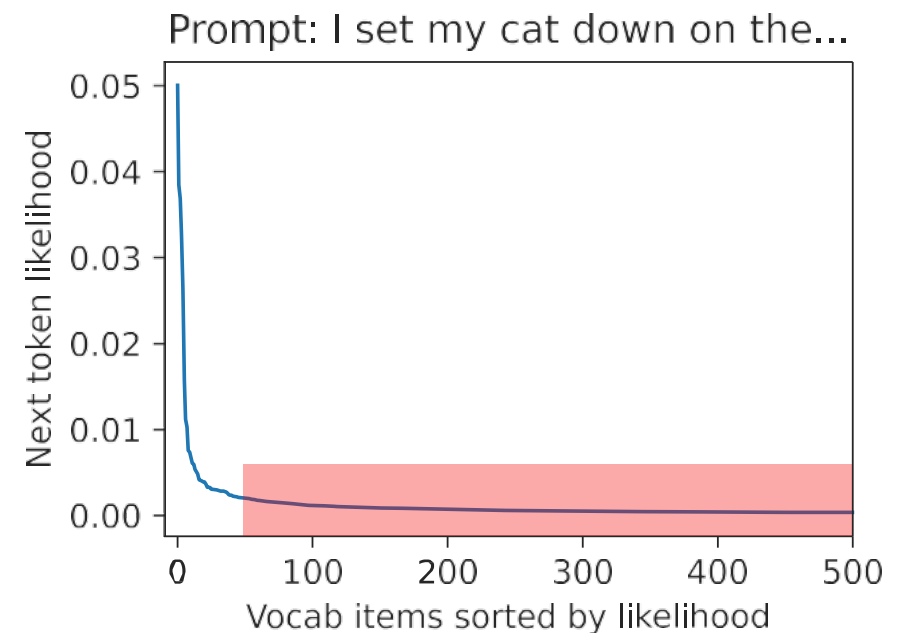
What would the probability of selecting “banana” be if we use temperature sampling and set $T = 0.00001$?

(a) 0% (b) 25% (c) 65% (d) 100%

How can we sample from $P(Y_t = i | y_{1:t-1})$?

- ❑ **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i | y_{1:t-1})$
- ❑ **Option 2:** Randomly sample from the distribution returned by the model.
- ❑ **Option 3:** Randomly sample with temperature.
- ❑ **Option 4:** Introduce sparsity by reassigning all probability mass to the k most likely tokens. This is referred to as top- k sampling.

Usually, k between 10 and 50 is selected.



How can we sample from $P(Y_t = i|y_{1:t-1})$?

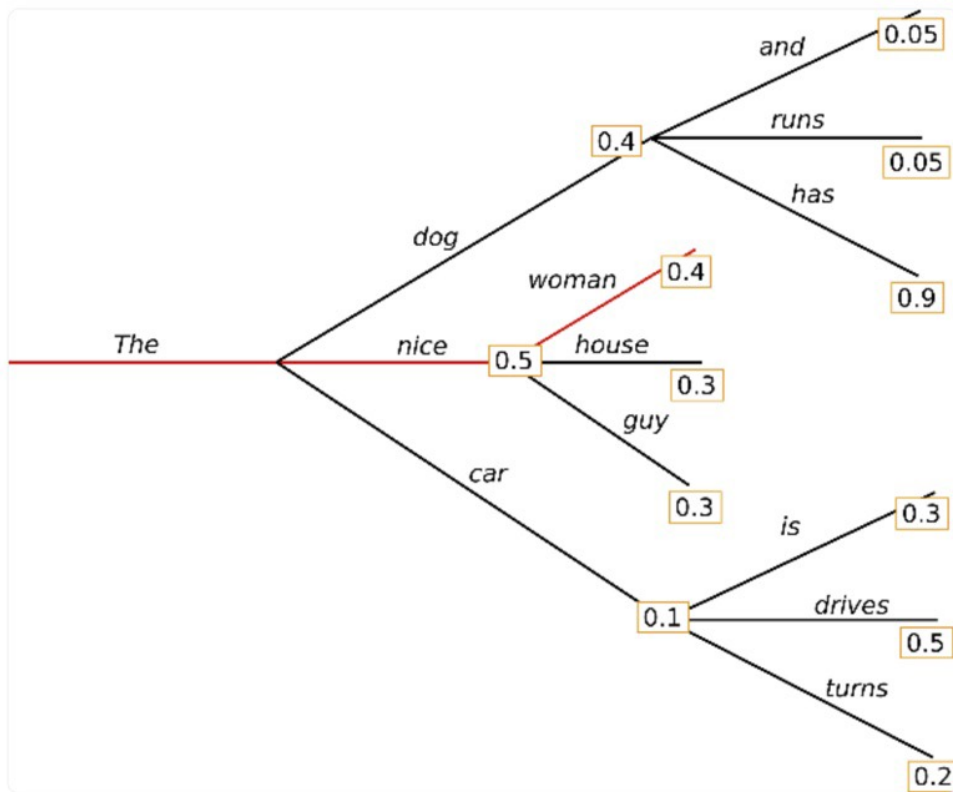
- ❑ **Option 1:** Take $\operatorname{argmax}_i P(Y_t = i|y_{1:t-1})$
- ❑ **Option 2:** Randomly sample from the distribution returned by the model.
- ❑ **Option 3:** Randomly sample with temperature.
- ❑ **Option 4:** Introduce sparsity by reassigning all probability mass to the k most likely tokens. This is referred to as top- k sampling.
- ❑ **Option 5:** Introduce sparsity by reassigning all probability mass to the k_t tokens which form $p\%$ of the probability mass. At each step, k is chosen such that the total probability of the k_t most likely tokens is no greater than the desired probability p . This is referred to as *nucleus sampling or top- p sampling*.
- ❑ **Option 6:** Use *beam search*.

Beam Search

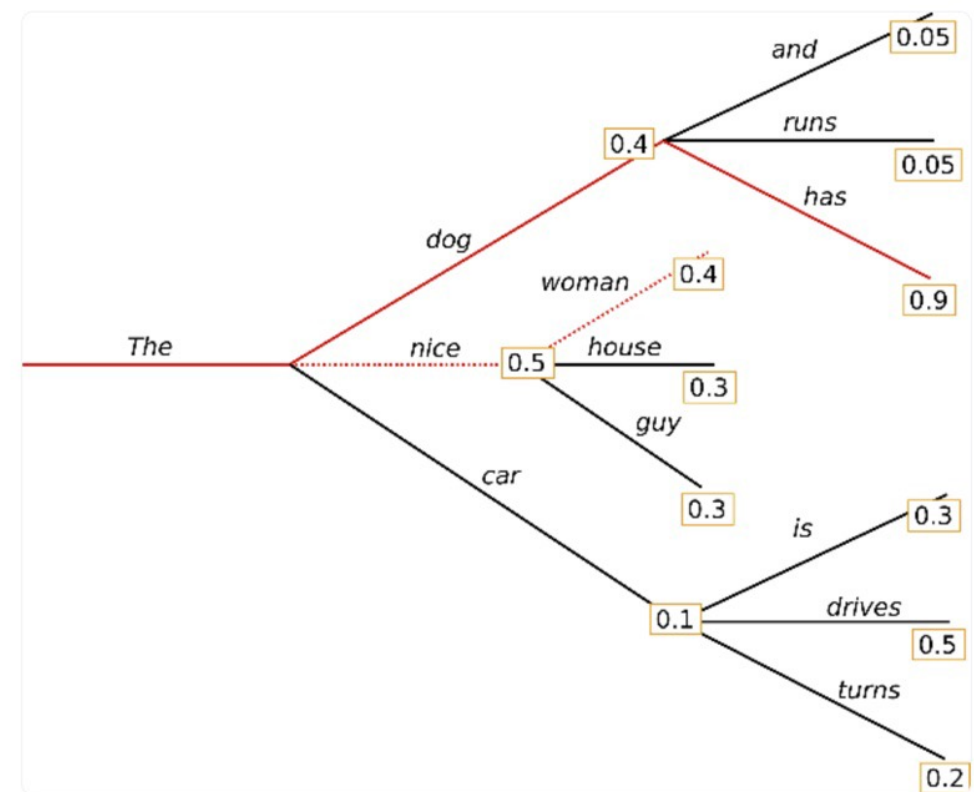
- ❑ **Assumption:** The best possible sequence to generate is the one with highest overall sequence likelihood (according to the model).
- ❑ It is computationally intractable to search all possible sequences for the most likely one, so instead, we use *beam search*.
- ❑ Beam search is a search algorithm that approximates finding the overall most likely sequence to generate.

Beam Search

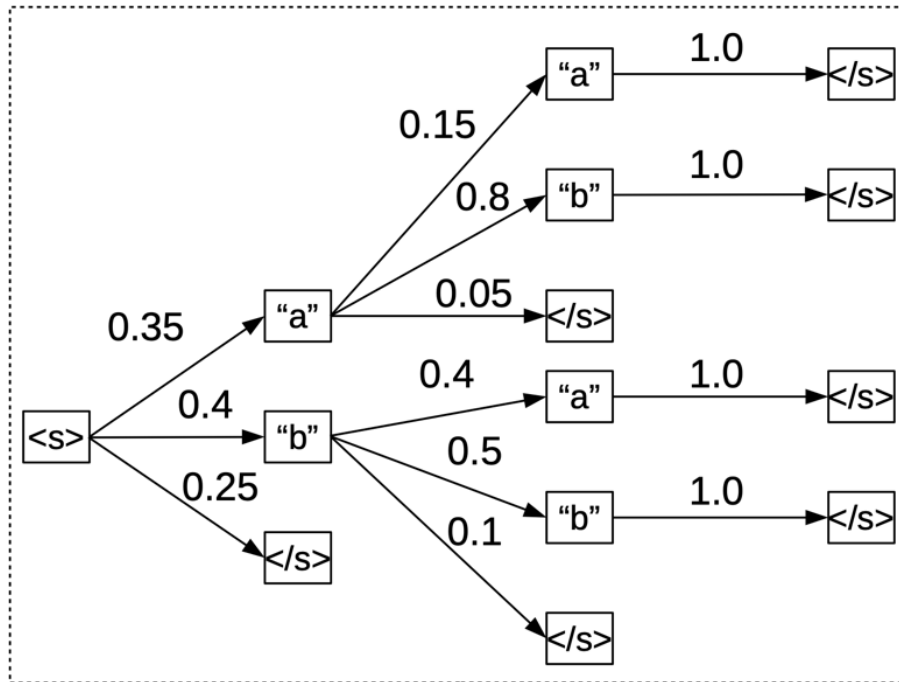
Greedy



Beam



Greedy search methods do not always lead to the most likely output.



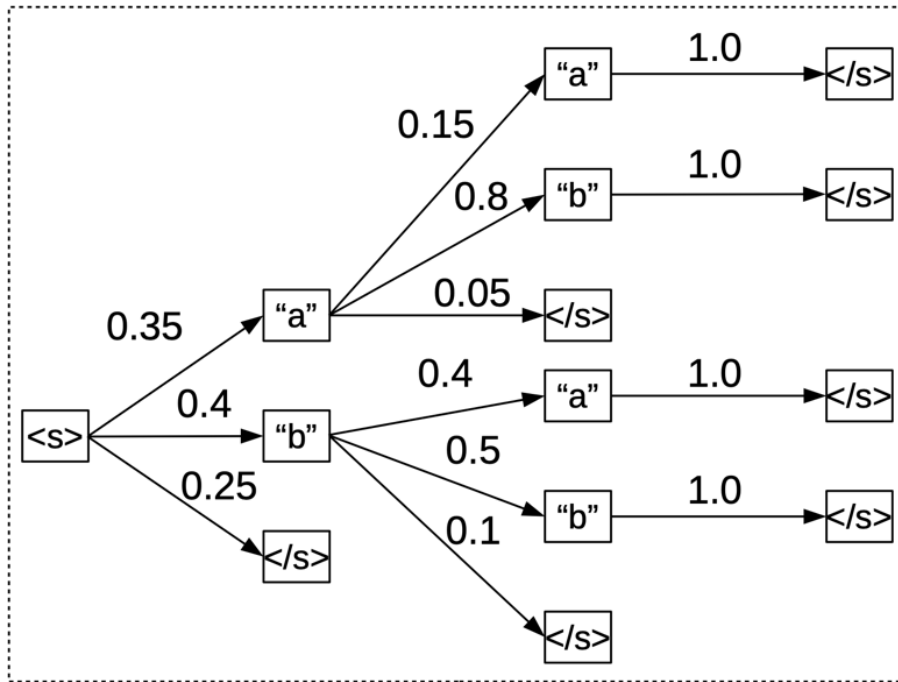
Vocabulary = {a, b, </s>}

Numbers above each edge are the transition probabilities $P(x_t|x_{1:t-t})$

If we were to choose the sequence that maximizes $P(x_1, \dots, x_T)$, which of the following would get generated?

- (a) [a, b, </s>]
- (b) [a, a, </s>]
- (c) [b, b, </s>]
- (d) [b, a, </s>]

Greedy search methods do not always lead to the most likely output.



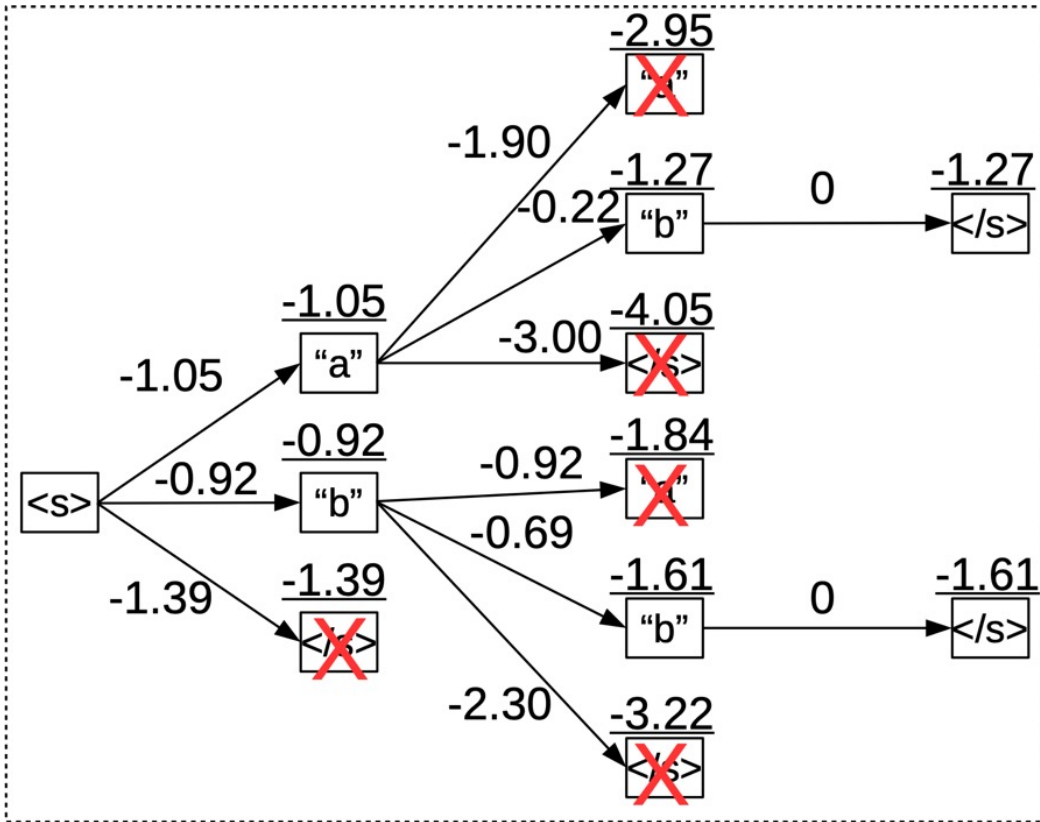
Vocabulary = {a, b, </s>}

Numbers above each edge are the transition probabilities $P(x_t|x_{1:t-t})$

If we were to choose the sequence that maximizes $P(x_1, \dots, x_T)$, which of the following would get generated?

- (a) [a, b, </s>]
- (b) [a, a, </s>]
- (c) [b, b, </s>]
- (d) [b, a, </s>]

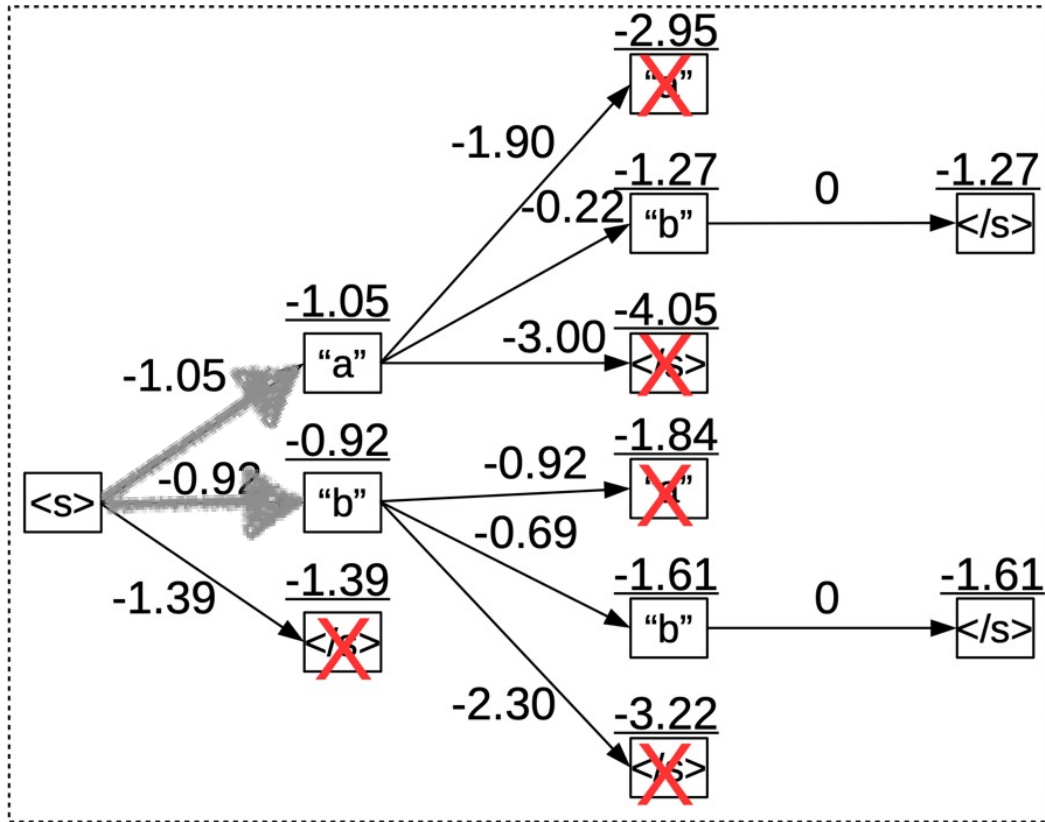
Beam search explores multiple possible output sequences, trying to find the overall most likely one.



Score each path and keep the top 2

- Vocabulary = {a, b, </s>}
- Numbers above the boxes are $\log P(x_t|x_{1:t-1})$
- Numbers shown on edges are $\log P(x_1, \dots, x_t)$
- Suppose we use beam search with a **beam size of 2**.

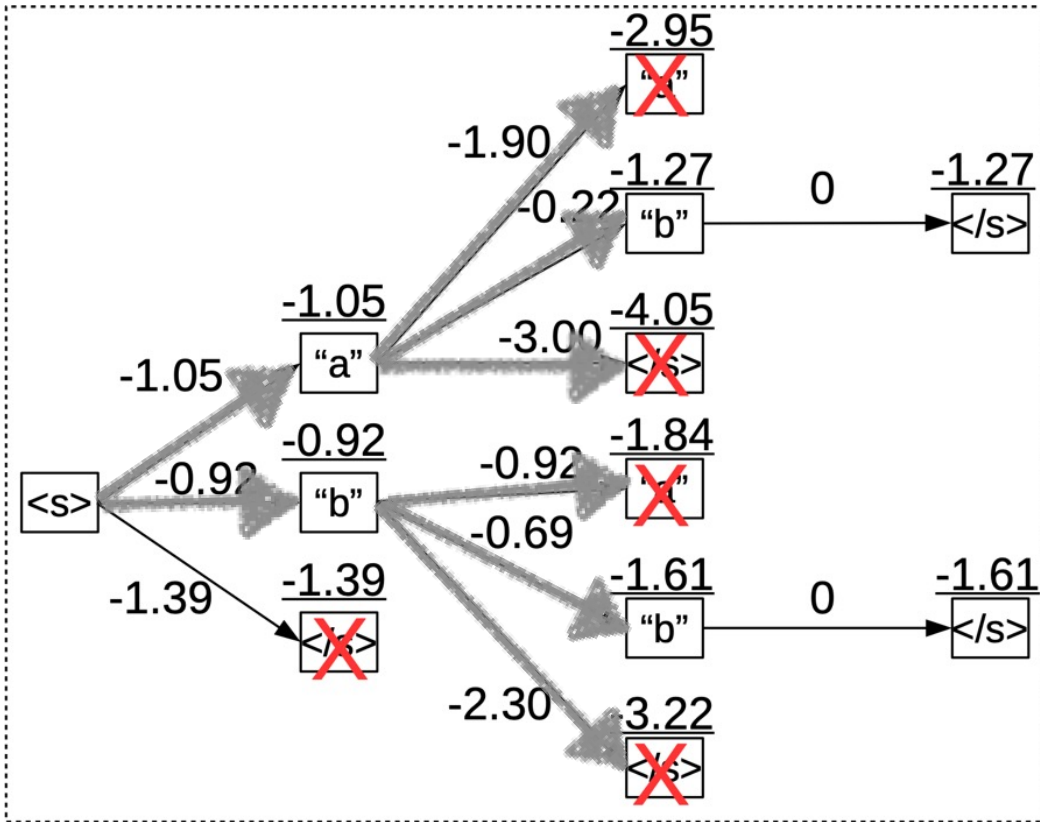
Beam search explores multiple possible output sequences, trying to find the overall most likely one.



Score each path and keep the top 2

- Vocabulary = {a, b, </s>}
- Numbers above the boxes are $\log P(x_t|x_{1:t-1})$
- Numbers shown on edges are $\log P(x_1, \dots, x_t)$
- Suppose we use beam search with a **beam size of 2**.

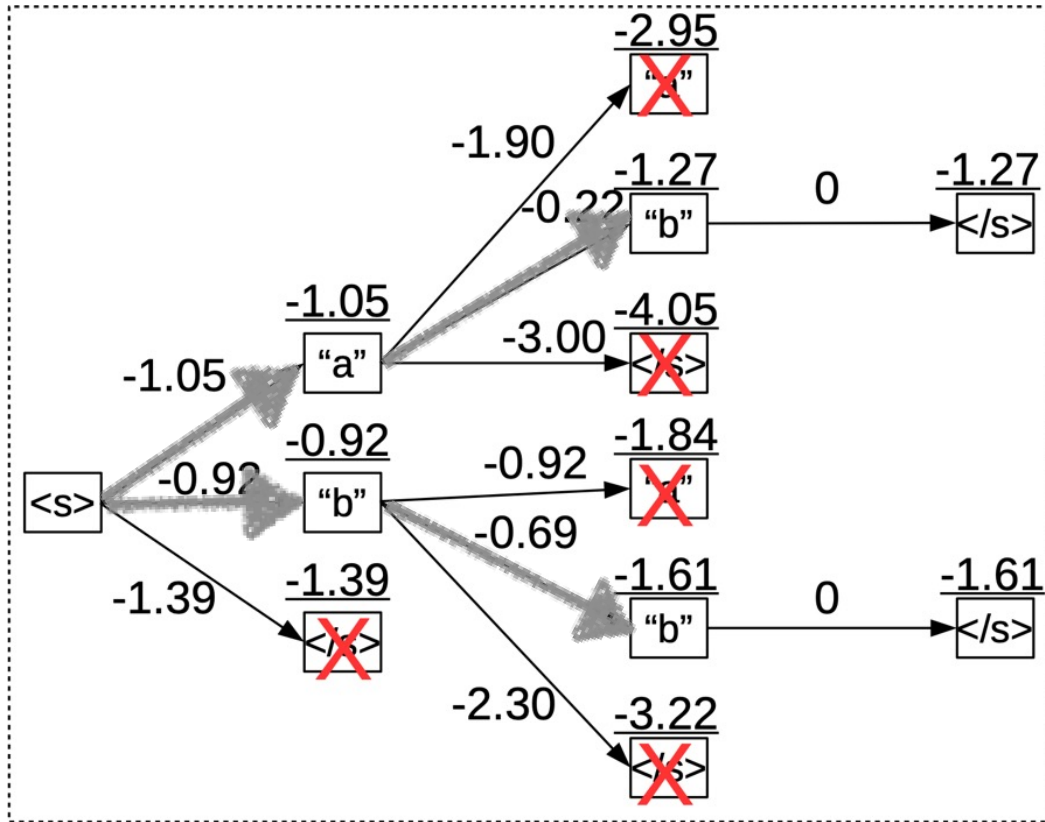
Beam search explores multiple possible output sequences, trying to find the overall most likely one.



Score each path and keep the top 2

- Vocabulary = {a, b, </s>}
- Numbers above the boxes are $\log P(x_t|x_{1:t-1})$
- Numbers shown on edges are $\log P(x_1, \dots, x_t)$
- Suppose we use beam search with a **beam size of 2**.

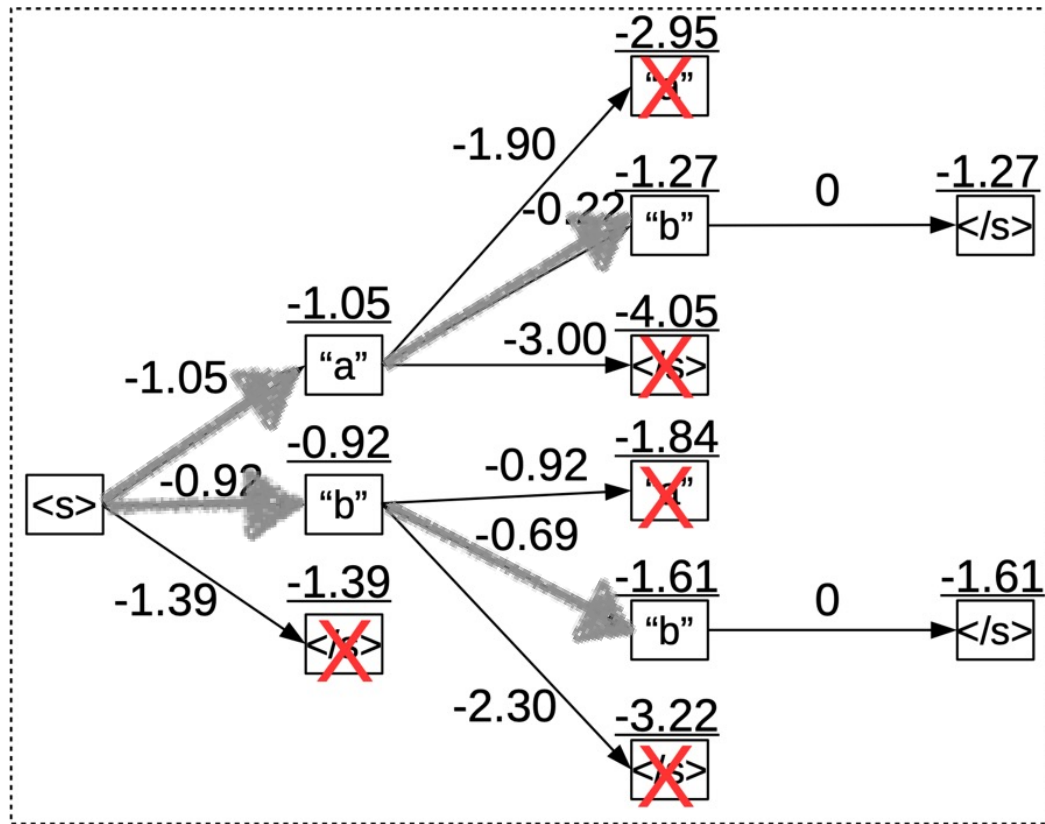
Beam search explores multiple possible output sequences, trying to find the overall most likely one.



Score each path and keep the top 2

- Vocabulary = {a, b, </s>}
- Numbers above the boxes are $\log P(x_t|x_{1:t-1})$
- Numbers shown on edges are $\log P(x_1, \dots, x_t)$
- Suppose we use beam search with a **beam size of 2**.

Beam search explores multiple possible output sequences, trying to find the overall most likely one.

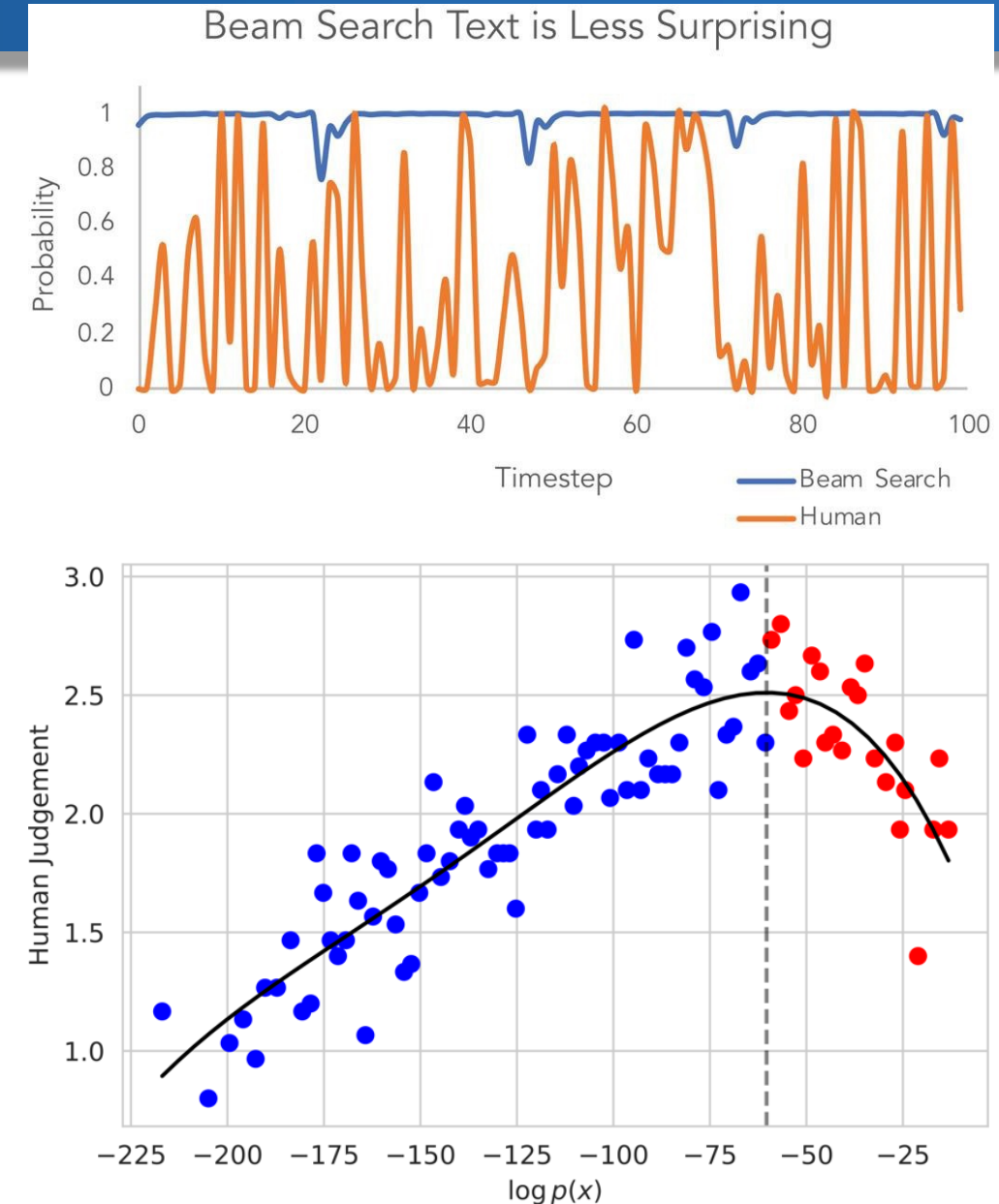


Score each path and keep the top 2

- Vocabulary = {a, b, </s>}
- Numbers above the boxes are $\log P(x_t|x_{1:t-1})$
- Numbers shown on edges are $\log P(x_1, \dots, x_t)$
- Suppose we use beam search with a **beam size of 2**.
- The paths that are still open at any step of the beam search algorithm are called **beams**.

Problems with Beam Search

- It turns out for open-ended tasks like dialog or story generation, optimizing for the sequence with the highest possible $P(x_1, \dots, x_T)$ isn't actually a great idea.
 - Beam search generates text that is much more likely than human-written text
 - When sequence likelihood is too high, humans rate text as bad.



Beam Search Applications

❑ When to Use Beam Search

- You need high-probability, deterministic output
 - Machine translation, speech recognition, OCR correction
- Evaluation is likelihood-based
 - BLEU in translation
- You want stable, repeatable results
- The output space is structured
 - Code generation (when correctness matters), formal text generation, summarization (sometimes)
- You are using a language model that isn't very good, and you don't trust its predicted probabilities.

❑ When **NOT** to use beam search:

- Creative writing, story generation, dialogue systems, and brainstorming ideas

Likelihood vs. Diversity



Other generation parameters you'll encounter

- ❑ **Frequency penalty:** Reduce the likelihood the model generates a token based on how often it has occurred already.
 - The more likely a token has occurred, the less likely it will occur in the future.
- ❑ **Presence penalty:** Reduce the likelihood the model generates a token based on whether or not it has occurred already.
 - If a token occurs any number of times, it will be less likely to occur in the future.
- ❑ **Stopping criteria**
 - Stop after generating k tokens.
 - Stop when a certain token is generated (for example, a period or a newline).

Smoothing

Training set:

- ... denied the allegations
- ... denied the reports
- ... denied the claims
- ... denied the request

Test set:

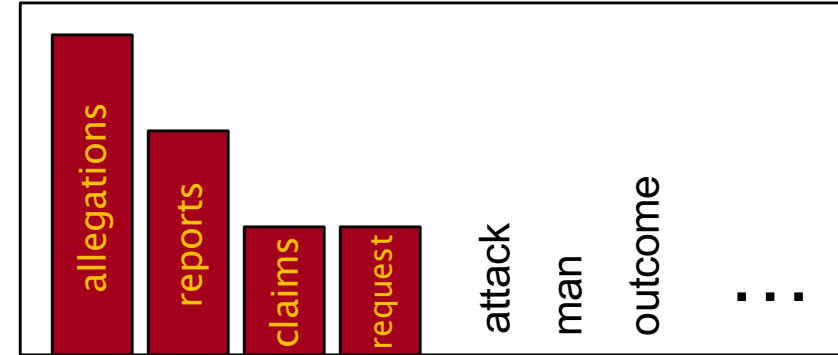
- ... denied the offer
- ... denied the loan

$$P(\text{"offer"} \mid \text{denied the}) = 0$$

The intuition of smoothing

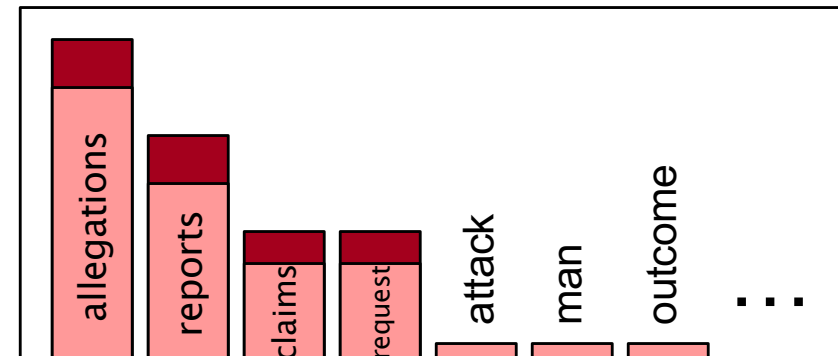
□ When we have sparse statistics:

- $P(w \mid \text{denied the})$
- 3 allegations
- 2 reports
- 1 claims
- 1 request
- 7 total



□ Steal probability mass to generalize better

- $P(w \mid \text{denied the})$
- 2.5 allegations
- 1.5 reports
- 0.5 claims
- 0.5 request
- 2 other
- 7 total



Problems with n-gram models

Sparsity Problem 1

Problem: What if “students opened their w_j ” never occurred in data? Then w_j has probability 0!

(Partial) Solution: Add small δ to count for every $w_j \in V$. This is called *smoothing*.

$$\square P(w_j | \text{students opened their}) = \frac{\text{count}(\text{students opened their} \mid w_j)}{\text{count}(\text{students opened their})}$$

Problems with n-gram models

Storage: Need to store count for all possible n -grams. So, model size is $O(\exp(n))$.

$$P(w_j | \text{students opened their}) = \frac{\text{count}(\text{students opened their} \mid w_j)}{\text{count}(\text{students opened their})}$$

Increasing n makes model size huge!

Another issue

- ❑ We treat all words / prefixes independently of each other!
 - students opened their _____
 - pupils opened their _____
 - scholars opened their _____
 - undergraduates opened their _____
 - students turned the pages of their _____
 - students attentively perused their _____
 - ...

Shouldn't we *share* information across these semantically-similar prefixes?

One-hot Vectors

- ❑ N-gram models rely on the “bag-of-words” assumption
- ❑ Represent each word/n-gram as a vector of zeros with a single 1 identifying its index in the vocabulary

vocabulary

i
hate
love
the
movie
film

movie = $\langle 0, 0, 0, 0, 1, 0 \rangle$

film = $\langle 0, 0, 0, 0, 0, 1 \rangle$

What are the issues
of representing a
word this way?

All words are equally (dis)similar!

movie = $\langle 0, 0, 0, 0, 1, 0 \rangle$

film = $\langle 0, 0, 0, 0, 0, 1 \rangle$

dot product is zero!

these vectors are **orthogonal**

What we want is a representation space in which words, phrases, sentences etc. that are semantically similar also have similar representations!

Enter neural networks!

Students opened their



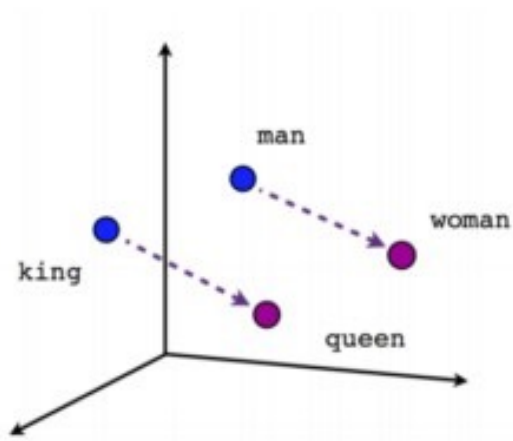
neural
language
model



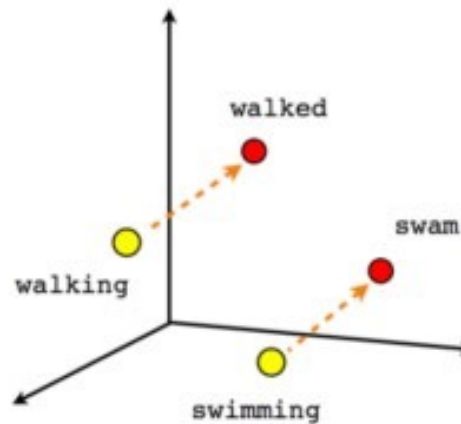
books

Words as basic building blocks

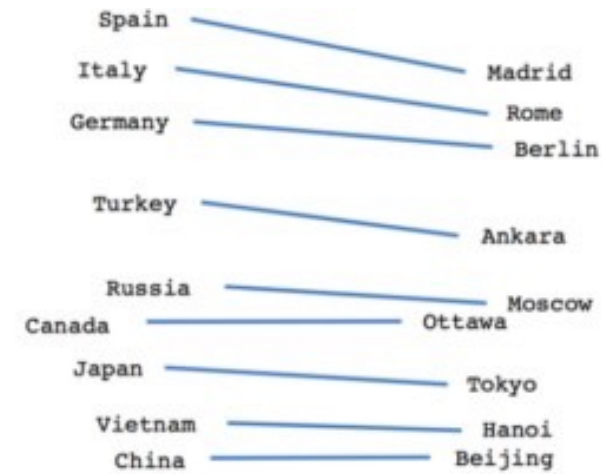
- Represent words with low-dimensional vectors called embeddings (Mikolov et al., NIPS 2013)
- king = [0.23, 1.3, -0.3, 0.43]



Male-Female



Verb tense



Country-Capital

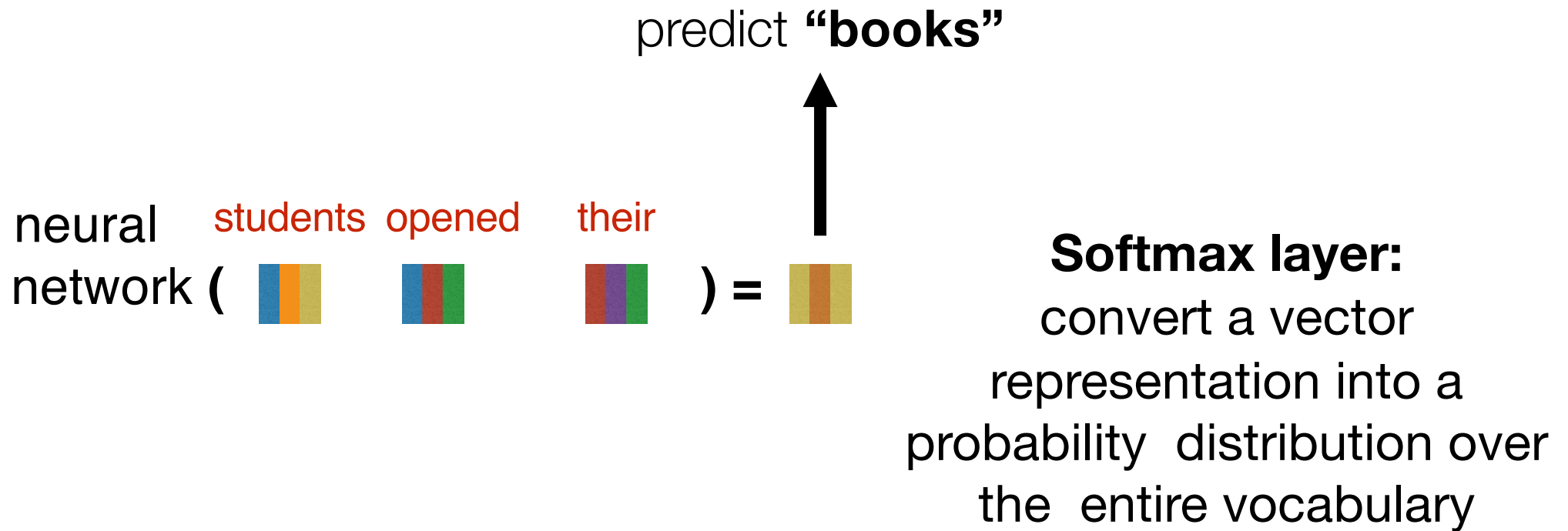
Composing embeddings

- Neural networks **compose** word embeddings into vectors for phrases, sentences, and documents.

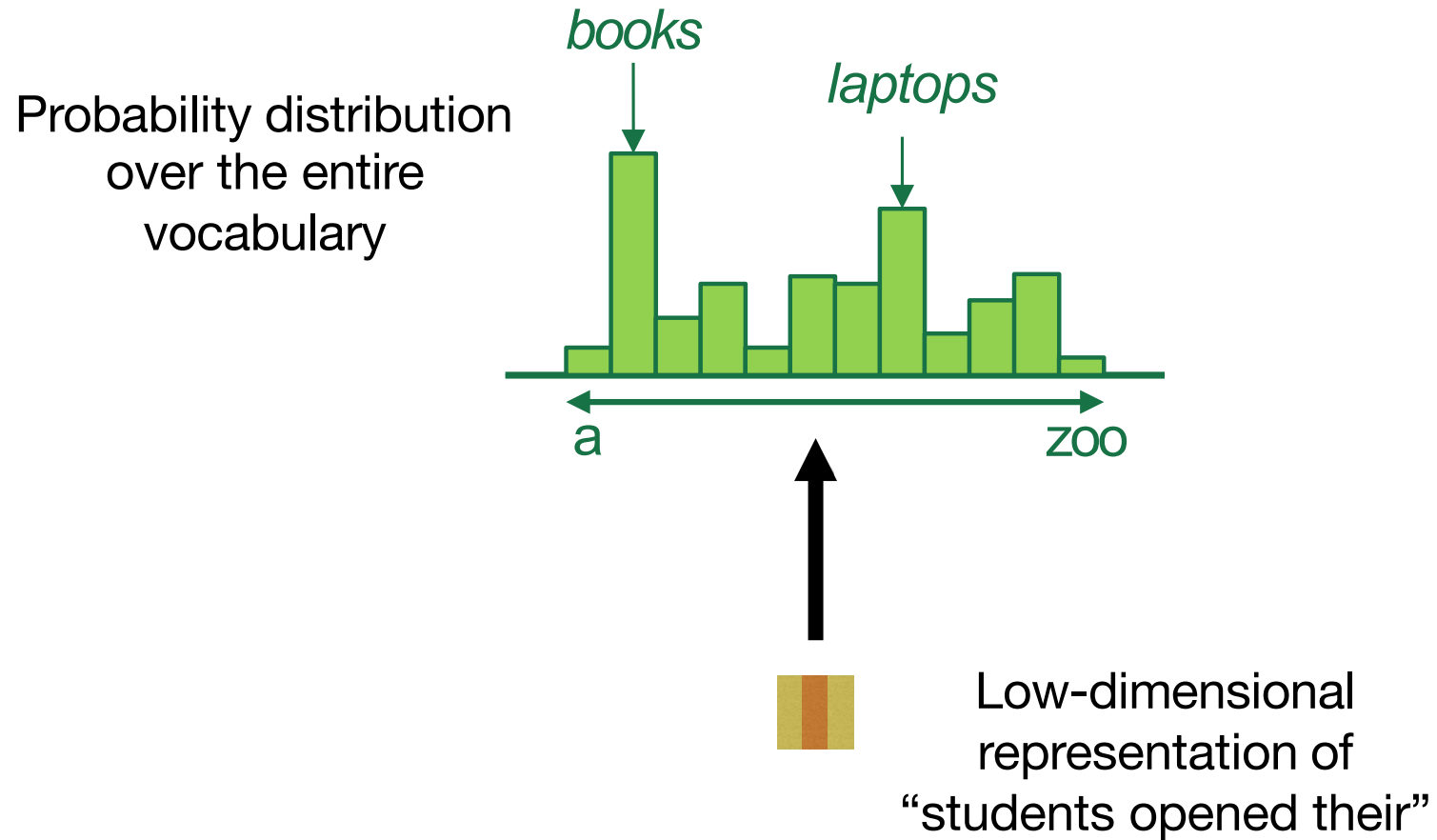
neural network (  ) = 

Next word prediction

- Predict the next word from composed prefix representation:



$P(w_i | \text{vector for "students opened their"})$



- Let's say our output vocabulary consists of just four words: “books”, “houses”, “lamps”, and “stamps”.

books houses lamps stamps
 $\langle 0.6, 0.2, 0.1, 0.1 \rangle$

We want to get a probability distribution over these four words



Low-dimensional representation of “students opened their”

$$x = [-2.3 \quad 0.9 \quad 5.4]$$

Here's an example 3-d
prefix vector



W is a *weight matrix*. It contains *parameters* that we can *update* to control the final probability distribution of the next word

$$\mathbf{W} = \begin{bmatrix} 1.2 & -0.3 & 0.9 \\ 0.2 & 0.4 & -2.2 \\ 8.9 & -1.9 & 6.5 \\ 4.5 & 2.2 & -0.1 \end{bmatrix}$$

$$\mathbf{x} = [-2.3 \quad 0.9 \quad 5.4]$$

Here's an example 3-d prefix vector



$$\mathbf{W} = \begin{bmatrix} 1.2 & -0.3 & 0.9 \\ 0.2 & 0.4 & -2.2 \\ 8.9 & -1.9 & 6.5 \\ 4.5 & 2.2 & -0.1 \end{bmatrix} \begin{matrix} \text{books} \\ \text{houses} \\ \text{lamps} \\ \text{stamps} \end{matrix}$$

$\mathbf{W}$ is a *weight matrix*. It contains *parameters* that we can *update* to control the final probability distribution of the next word

first, we'll project our 3-d prefix representation to 4-d with a matrix-vector product

$$\mathbf{x} = [-2.3 \quad 0.9 \quad 5.4]$$

Here's an example 3-d prefix vector



intuition: each row
of **W** contains
feature weights for a
corresponding word
in the vocabulary

$$\mathbf{W} = \begin{bmatrix} 1.2 & -0.3 & 0.9 \\ 0.2 & 0.4 & -2.2 \\ 8.9 & -1.9 & 6.5 \\ 4.5 & 2.2 & -0.1 \end{bmatrix} \begin{matrix} \text{books} \\ \text{houses} \\ \text{lamps} \\ \text{stamps} \end{matrix}$$

$$\mathbf{x} = [-2.3 \quad 0.9 \quad 5.4]$$

CAUTION: we can't
easily interpret these
features! For example,
the second dimension
of $\mathbf{x}$ likely does not
correspond to any
linguistic property

intuition: each
dimension of $\mathbf{x}$
corresponds to a
feature of the prefix

$$Wx^T = [1.8 \quad -11.9 \quad 12.9 \quad -8.9]$$

How did we compute this? Just the dot product of each row of **W** with **x**!

$$W = \begin{bmatrix} 1.2 & -0.3 & 0.9 \\ 0.2 & 0.4 & -2.2 \\ 8.9 & -1.9 & 6.5 \\ 4.5 & 2.2 & -0.1 \end{bmatrix} \begin{matrix} \text{books} \\ \text{houses} \\ \text{lamps} \\ \text{stamps} \end{matrix}$$

$x = [-2.3 \quad 0.9 \quad 5.4]$

$1.2 * -2.3 + -0.3 * 0.9 + 0.9 * 5.4$

Okay, so how do we go from this 4-d vector to a probability distribution?

$$Wx^T = [1.8 \quad -11.9 \quad 12.9 \quad -8.9]$$

We'll use the softmax function!

$$\text{softmax}(x) = \frac{e^x}{\sum e^{x_j}}$$

- x is a vector
- x_j is dimension j of x
- Each dimension j of the softmaxed output represents the probability of class j

$$\mathbf{Wx} = \langle 1.8, -1.9, 2.9, -0.9 \rangle$$

$$\text{softmax}(\mathbf{Wx}) = \langle 0.24, 0.006, 0.73, 0.02 \rangle$$

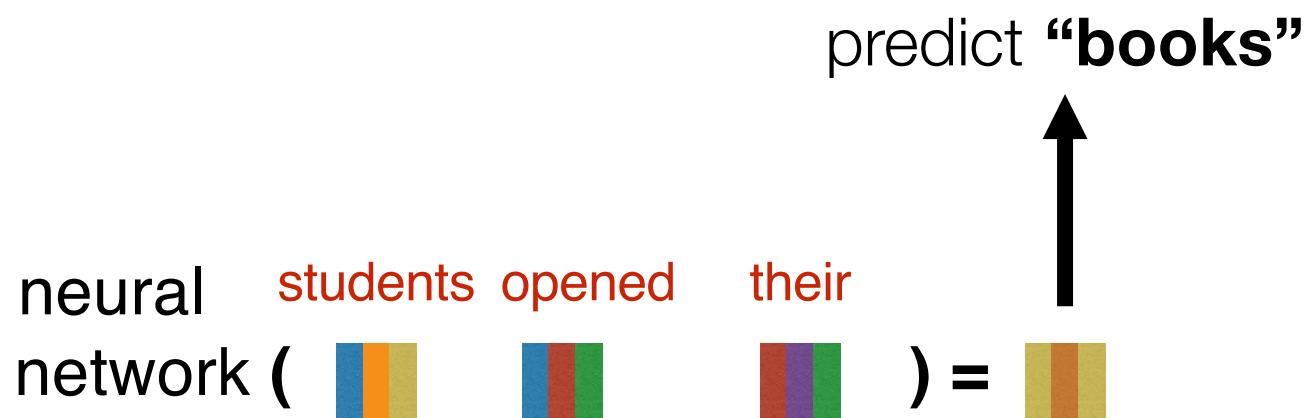
books houses lamps stamps

We'll see the softmax function over and over again this semester, so be sure to understand it!

So to sum up...

- Given a d -dimensional vector representation $\mathbf{x}$ of a prefix, we do the following to predict the next word:
 - Project it to a V -dimensional vector using a matrix-vector product (a.k.a. a “linear layer”, or a “feedforward layer”), where V is the size of the vocabulary.
 - Apply the softmax function to transform the resulting vector into a probability distribution.

- Now that we know how to predict “books”, let’s focus on how to compute the prefix representation $\mathbf{x}$ in the first place!



Composition functions

- ❑ **Input:** sequence of word embeddings corresponding to the tokens of a given prefix
- ❑ **Output:** single vector
- ❑ Composition functions:
 - Element-wise functions
 - e.g., just sum up all of the word embeddings!
 - Concatenation
 - Feed-forward neural networks
 - Convolutional neural networks
 - Recurrent neural networks
 - Transformers (our focus this semester)

- ❑ Let's look first at *concatenation*, an easy to understand but limited composition function.

A fixed-window neural Language Model

~~as the preter started the sleek~~ the students opened their _____

discard fixed window

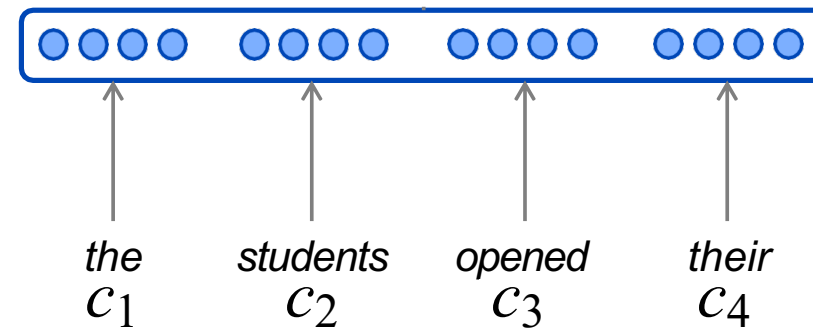
A fixed-window neural Language Model

concatenated word embeddings

$$x = [c_1; c_2; c_3; c_4]$$

words / one-hot vectors

c_1, c_2, c_3, c_4



A fixed-window neural Language Model

hidden layer

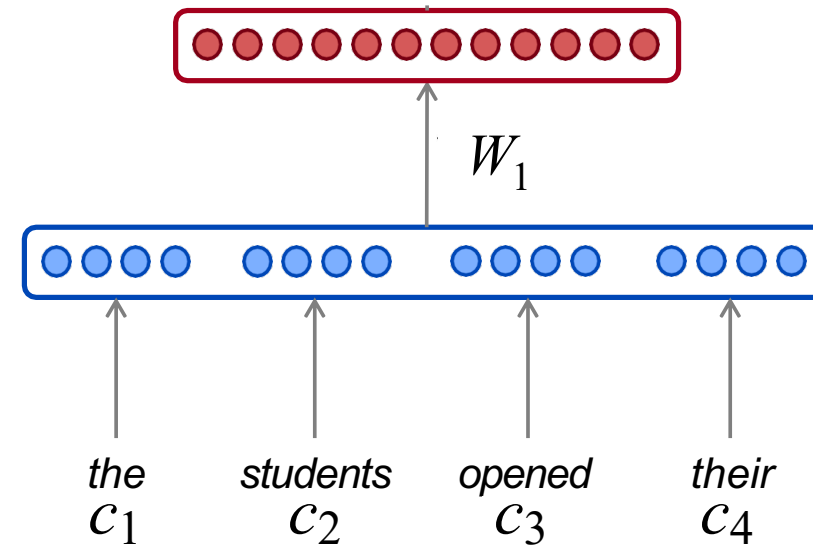
$$h = f(W_1 x)$$

concatenated word embeddings

$$x = [c_1; c_2; c_3; c_4]$$

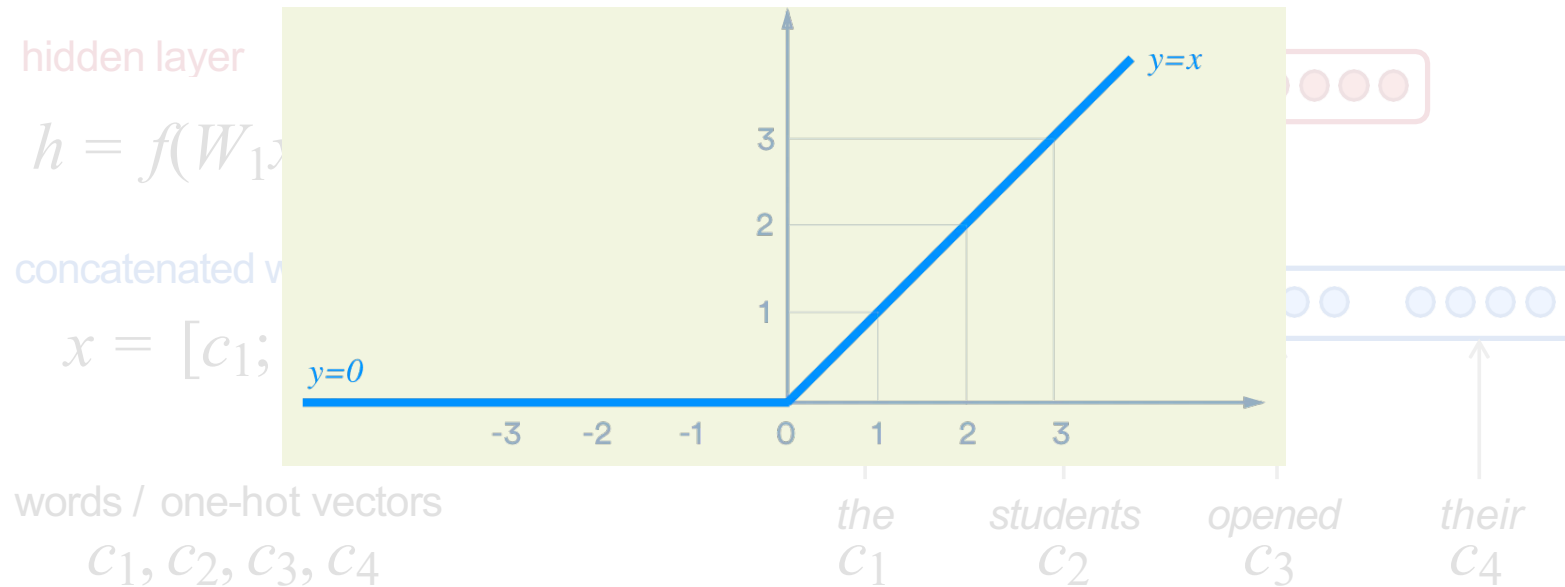
words / one-hot vectors

c_1, c_2, c_3, c_4



A fixed-window neural Language Model

f is a *nonlinearity*, or an element-wise nonlinear function.
The most commonly-used choice today is the rectified linear unit (**ReLu**), which is just $\text{ReLu}(x) = \max(0, x)$.
Other choices include **tanh** and **sigmoid**.



A fixed-window neural Language Model

output distribution

$$\hat{y} = \text{softmax}(W_2 h)$$

hidden layer

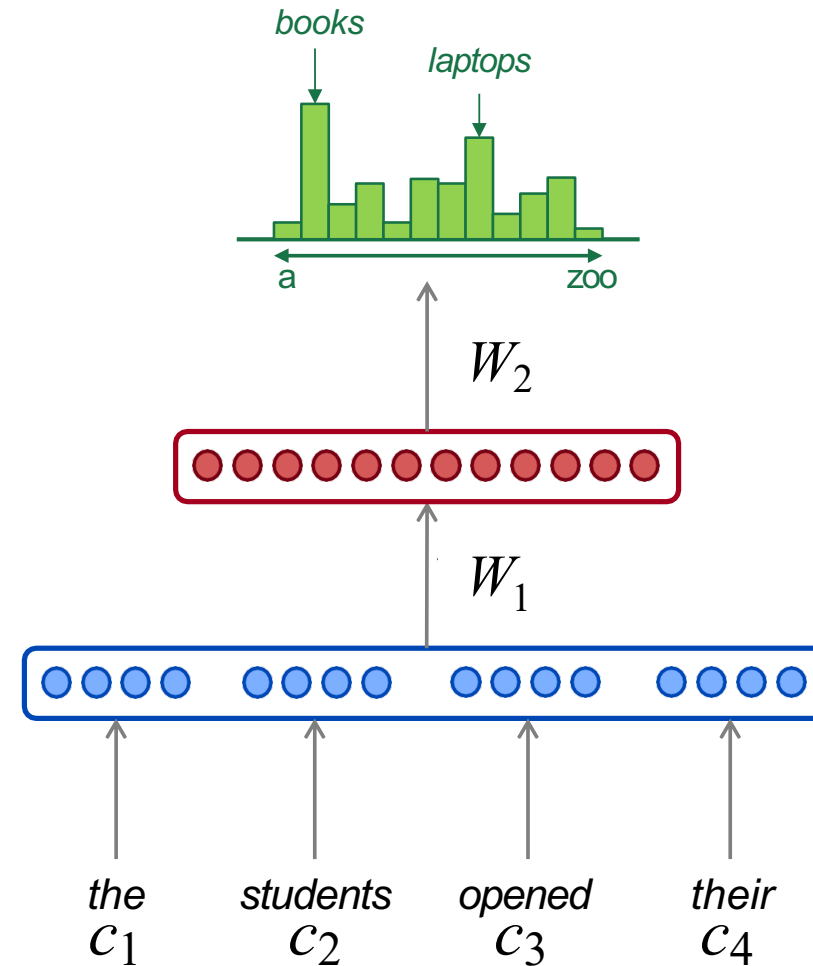
$$h = f(W_1 x)$$

concatenated word embeddings

$$x = [c_1; c_2; c_3; c_4]$$

words / one-hot vectors

$$c_1, c_2, c_3, c_4$$



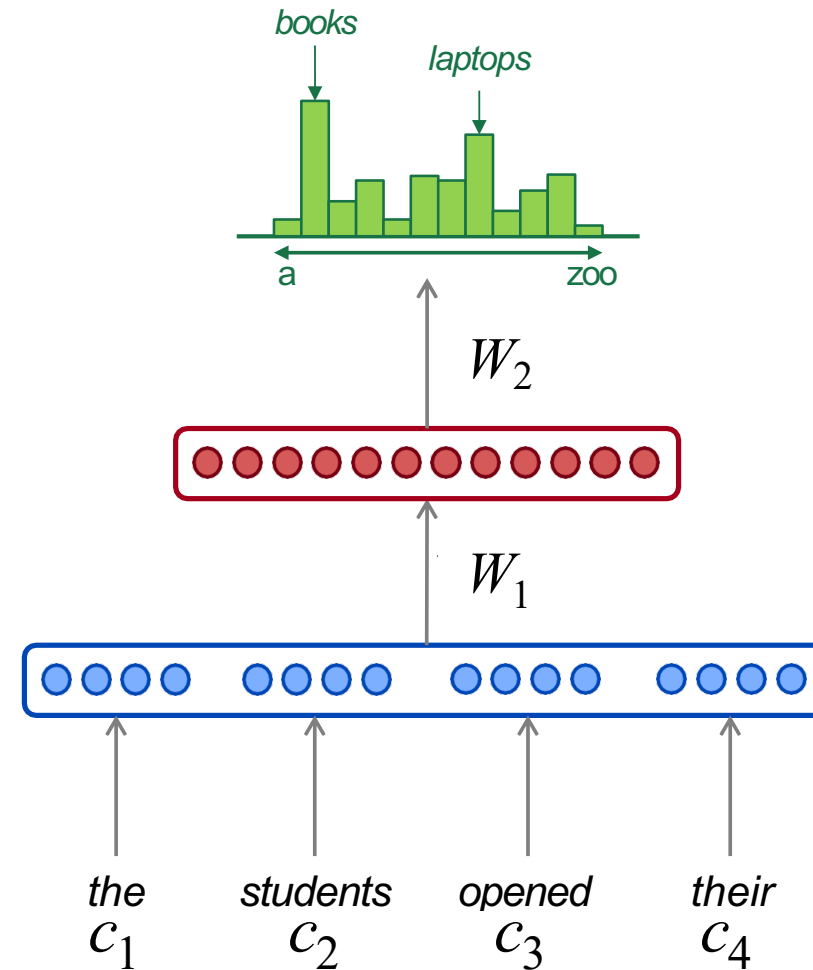
How does this compare to a normal n-gram model?

❑ Improvements over n -gram LM:

- No sparsity problem
- Model size is $O(n)$ not $O(\exp(n))$

❑ Remaining **problems**:

- Fixed window is **too small**
- Enlarging window enlarges W
- Window can never be large enough!
- Each uses different rows of W . We **don't share weights** across the window.



Recurrent Neural Networks!

Circa 2013: Recurrent neural networks

Generating Sequences With Recurrent Neural Networks

Alex Graves
Department of Computer Science
University of Toronto
`graves@cs.toronto.edu`

Abstract

This paper shows how Long Short-term Memory recurrent neural networks can be used to generate complex sequences with long-range structure, simply by predicting one data point at a time. The approach is demonstrated for text (where the data are discrete) and online handwriting (where the data are real-valued). It is then extended to handwriting synthesis by allowing the network to condition its predictions on a text sequence. The resulting system is able to generate highly realistic cursive handwriting in a wide variety of styles.

An RNN Language Model

output distribution

$$\hat{y} = \text{softmax}(W_2 h^{(t)})$$

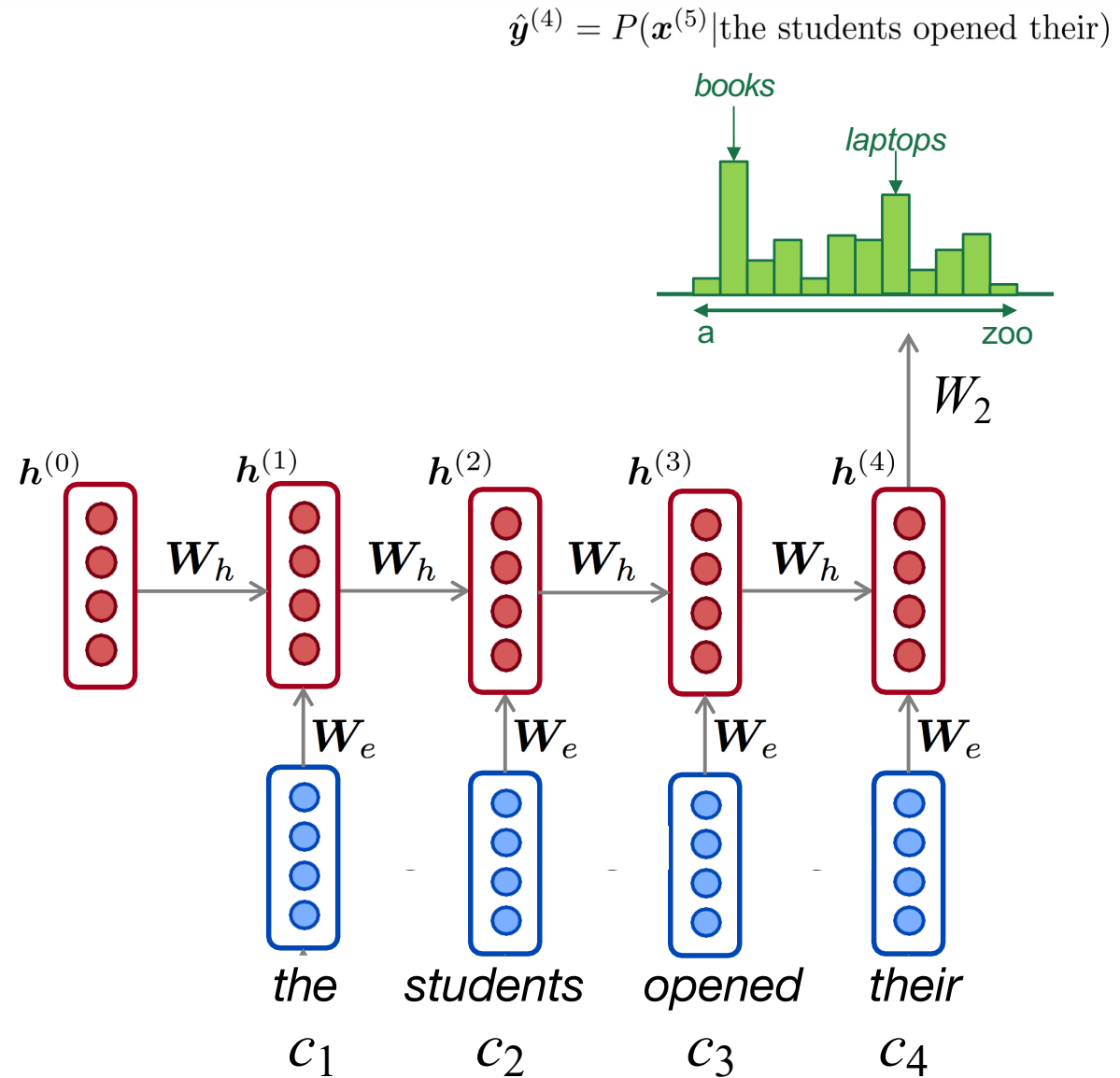
hidden states

$$h^{(t)} = f(W_h h^{(t-1)} + W_e c_t)$$

$h^{(0)}$ is initial hidden state!

word embeddings

c_1, c_2, c_3, c_4



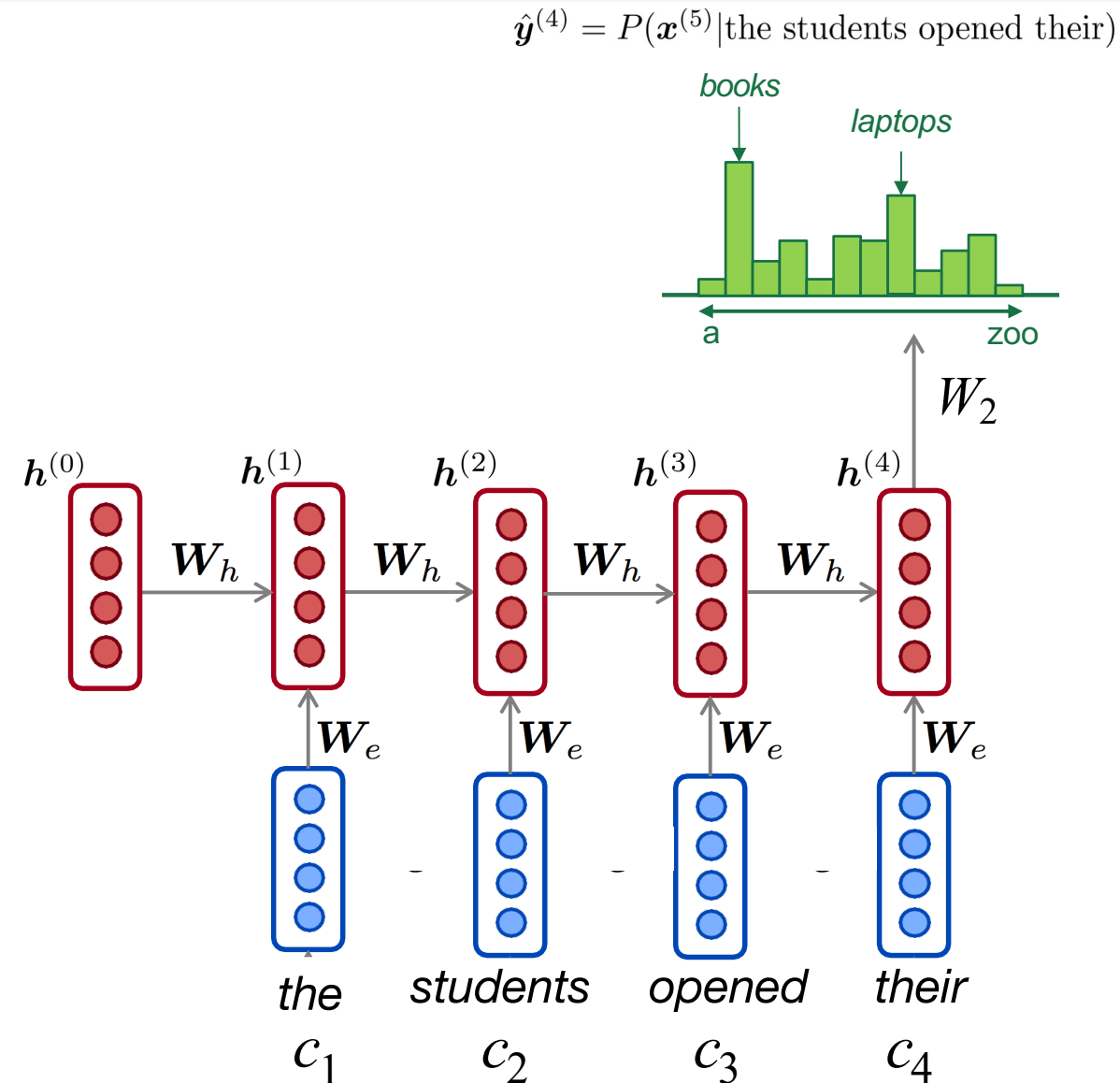
Why is this good?

❑ RNN Advantages:

- Can process **any length** input
- **Model size doesn't increase** for longer input
- Computation for step t can (in theory) use information from **many steps back**
- Weights are **shared** across timesteps $\rightarrow$ representations are shared

❑ RNN Disadvantages:

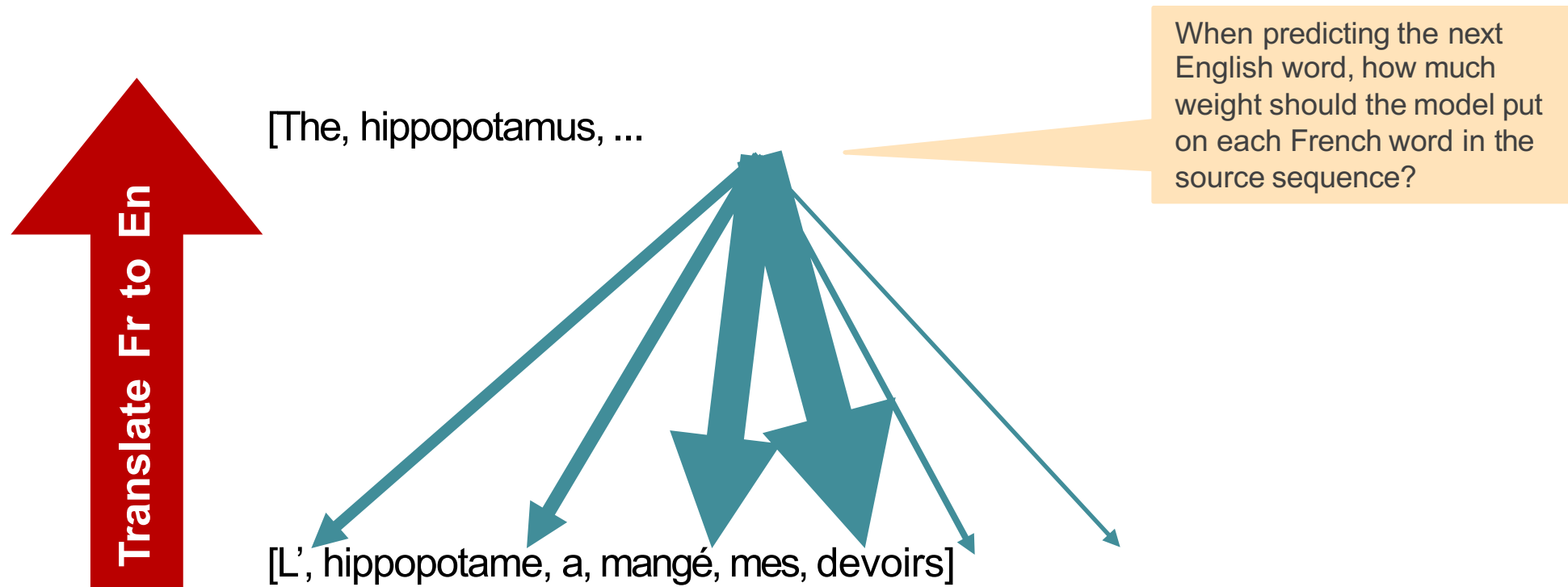
- Vanishing gradient
- Recurrent computation is **slow**
- In practice, difficult to access information from **many steps back**



Transformers

Connecting Encoder to Decoders in RNN

- Better approach: an attention mechanism.



Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate." (2014).

- At each step t in the decoder, a context vector is computed which contains all the information from the encoder that is relevant to the decoder making a prediction at this position.

Compute a linear combination of the encoder hidden states.

$$\mathbf{c}_t = \alpha_1 \mathbf{h}_1^{\text{enc}} + \alpha_2 \mathbf{h}_2^{\text{enc}} + \alpha_3 \mathbf{h}_3^{\text{enc}} + \dots + \alpha_T \mathbf{h}_T^{\text{enc}}$$

The context vector is a linear sum of the encoder hidden states, i.e., $\mathbf{c}_t = \mathbf{H}^{\text{enc}} \boldsymbol{\alpha}_t$.

- The decoder's predicted embedding for position t is a function of the context vector and the decoder's hidden state for this position.

Decoder's prediction at position t is based on both the context vector and the hidden state outputted by the RNN at that position.

$$\hat{\mathbf{e}}_t = f_{\theta}(\mathbf{h}_t^{\text{dec}} \parallel \mathbf{c}_t)$$

$$\hat{\mathbf{e}}_t = f_{\theta}(\mathbf{h}_t^{\text{dec}}; \alpha_{1,t} \mathbf{h}_1^{\text{enc}} + \alpha_{2,t} \mathbf{h}_2^{\text{enc}} + \dots + \alpha_{T,t} \mathbf{h}_T^{\text{enc}})$$

Computing the Attention Weights

- The $\alpha_{i,j}$ are scores that indicate how important the encoder hidden state at position i is to the model's prediction at position j .
- They are typically normalized to sum to 1.

$$\alpha_{i,j} = \frac{\exp e_{i,j}}{\sum_{k=1}^T \exp e_{i,k}} \quad \leftarrow \text{Softmax function}$$
$$e_{i,j} = \text{score}(h_i^{enc}, h_{j-1}^{dec})$$

- In dot-product attention, we use a very simple scoring function

$$\text{score}(\mathbf{q}, \mathbf{k}) = \mathbf{q} \cdot \mathbf{k}$$

- *“At the core of an attention-based approach is the ability to compare an item of interest to a collection of other items in a way that reveals their relevance in the current context.”* Jurafsky and Martin, Chapter 10.

Circa 2017: Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

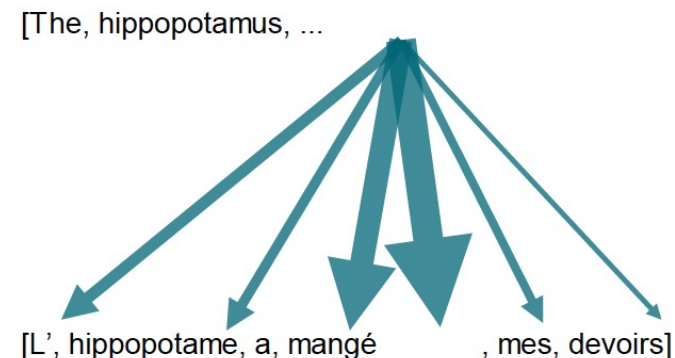
Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

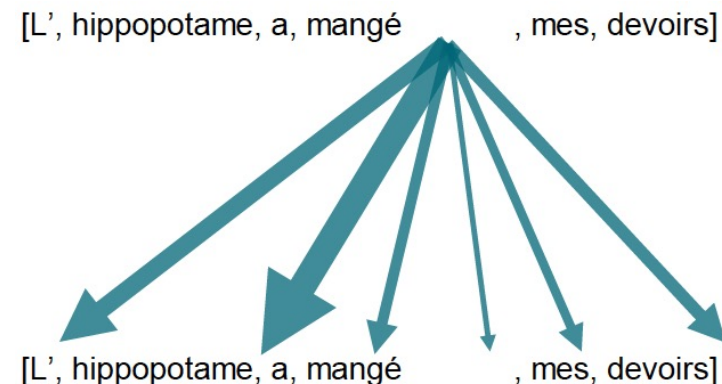
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Encoder-decoder attention:



Self-attention:



Why drop the recurrence and only use attention?

- ❑ Recurrent neural networks are *slow to train*. Computation cannot be parallelized.
 - The computation at position t is dependent on first doing the computation at position $t - 1$.
- ❑ Recurrent neural networks do *poorly with long contexts*.
 - If two tokens are k positions apart, there are k opportunities for knowledge of the first token to be erased from the hidden state before a prediction is made at the position of the second token.
- ❑ Transformers solve both these problems.

Components of a Generic Attention Mechanism

- A sequence of <key, value> embeddings pairs
 - The values are always the hidden states from a previous layer of the neural network. The attention mechanism outputs a weighted sum of these.
 - For encoder-decoder attention, the values are the final hidden states of the encoder (as we saw in the previous slide) and the keys are the hidden states from the target sequence.
- A sequence of query embeddings
 - The query is the current focus of the attention.
 - We choose weights for each of the values by computing a score between the current query and each of the keys.

Attention output at position $j = \sum_{i=1}^T \text{score}(q_j, k_i) \cdot v_i$

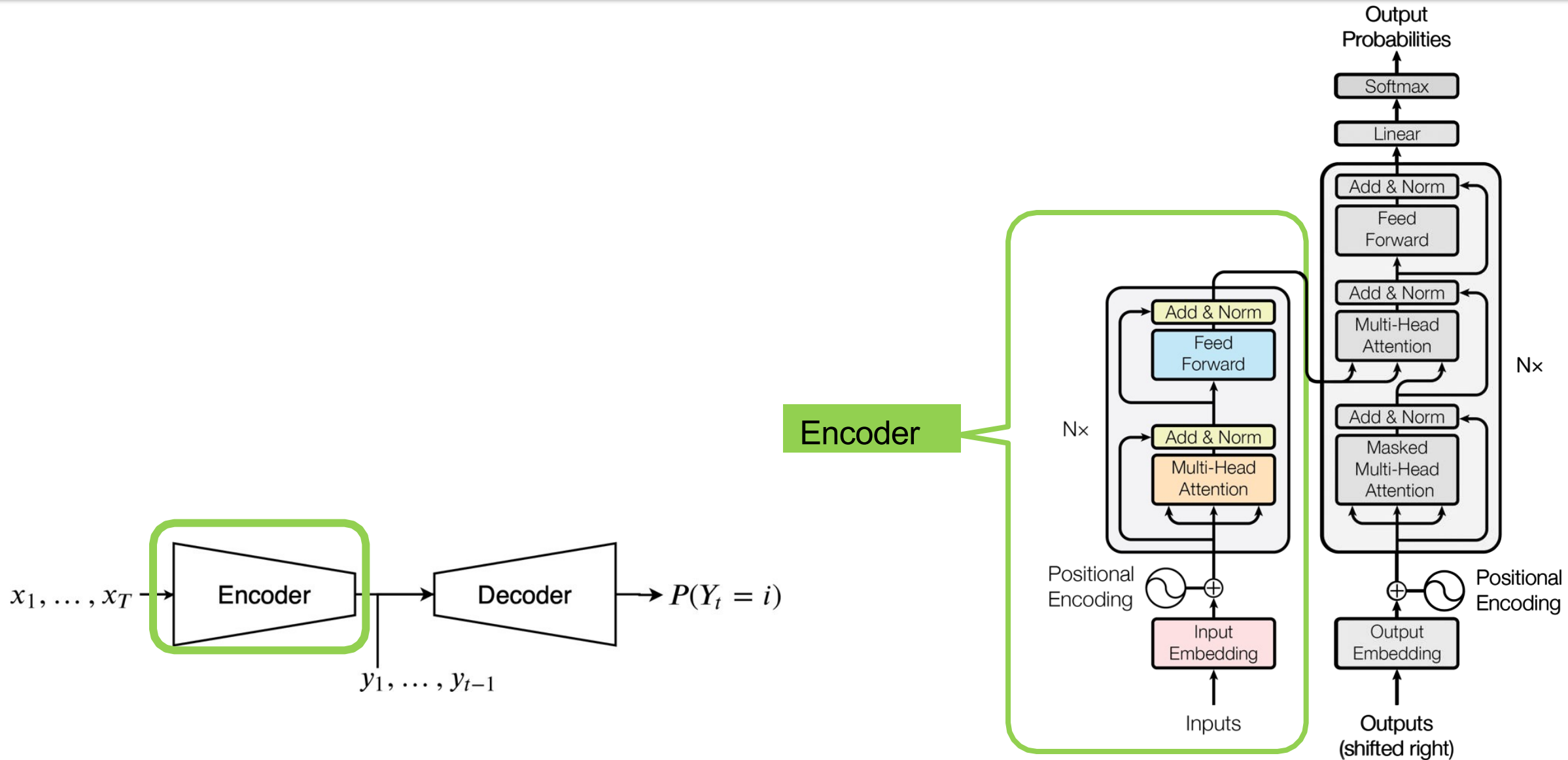
$$\text{score}(q_j, k_i) = \frac{q_j \cdot k_i}{\sqrt{d_k}}$$

Components of a Generic Attention Mechanism

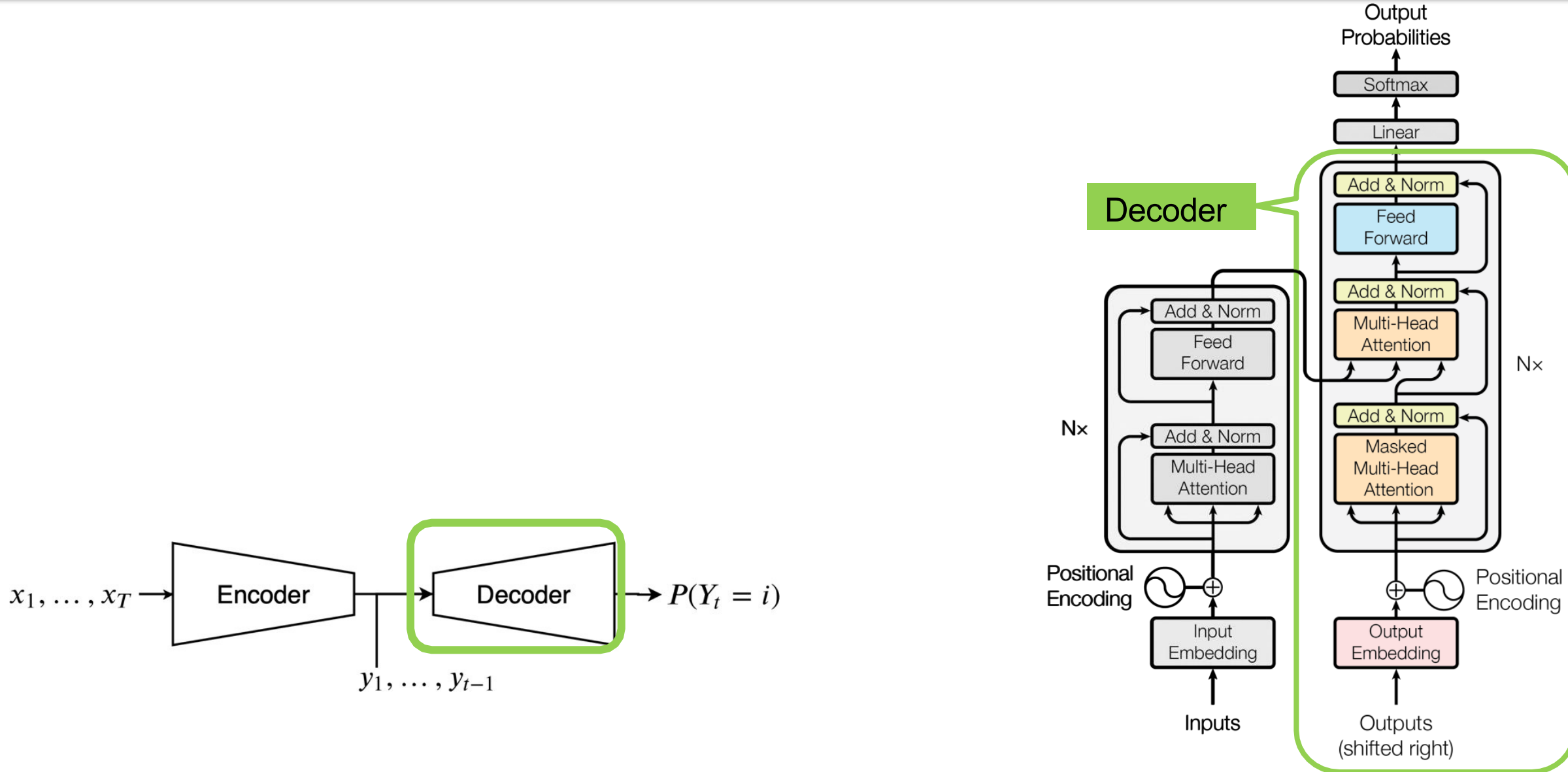
- Since the attention computations at each position j are completely independent, we can actually parallelize all these computations and think in terms of matrix multiplications.
- For example, instead of thinking of a sequence of embedding vectors $\mathbf{x}_1, \dots, \mathbf{x}_T$, we can think of a matrix $\mathbf{X} \in \mathbb{R}^{T \times d_x}$.
- This gives us the attention equation which appear in the “*Attention is All You Need*” paper.

$$\text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

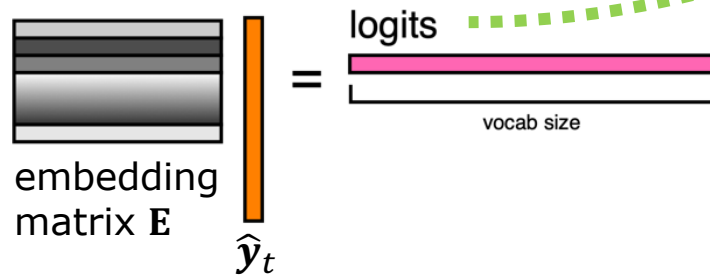
Transformers: “Attention is All You Need”



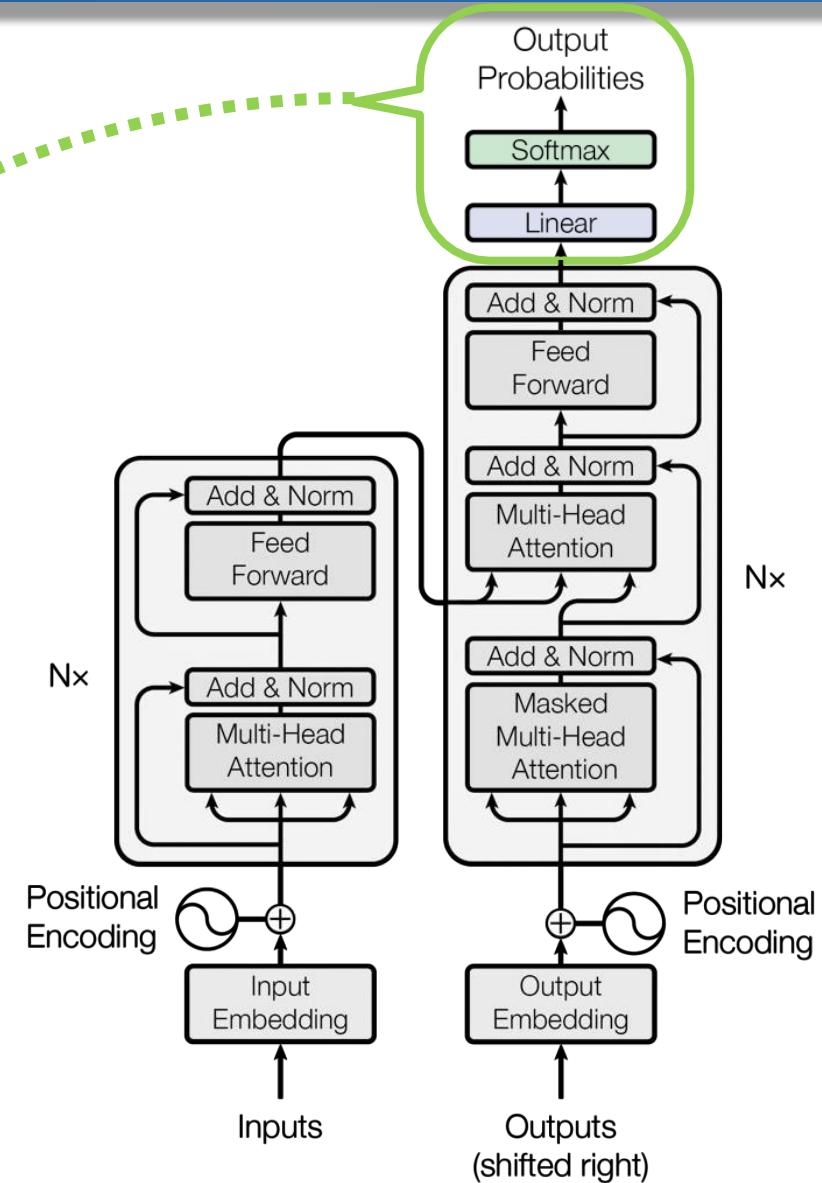
Transformers: “Attention is All You Need”



Transformers: “Attention is All You Need”

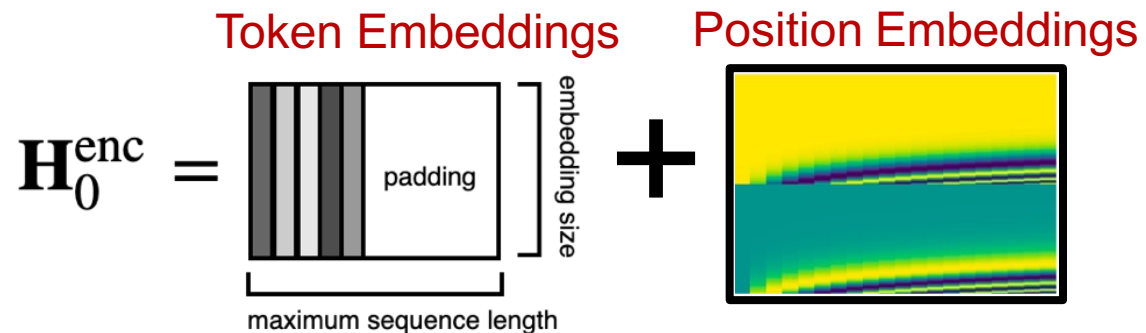


$$P(Y_t = i | \mathbf{x}_{1:T}, \mathbf{y}_{1:t-1}) = \frac{\exp(E\hat{y}_t[i])}{\sum_j \exp(E\hat{y}_t[j])}$$

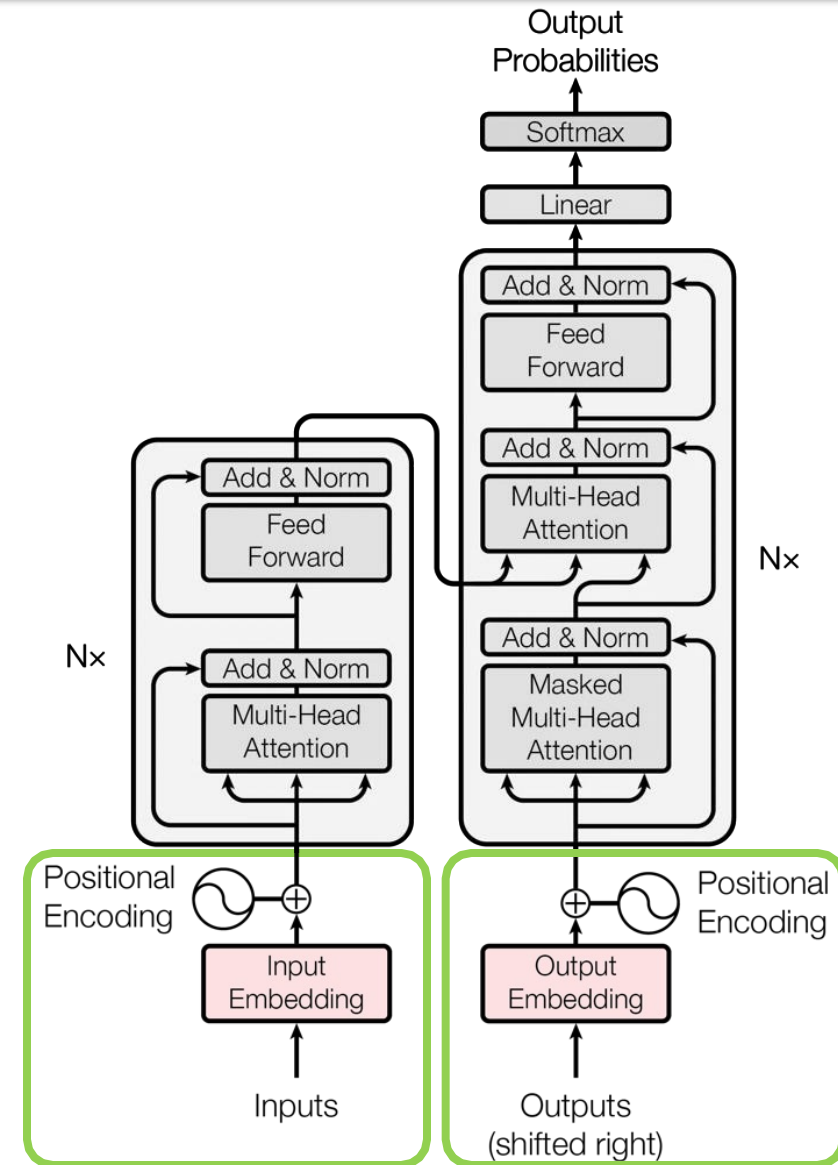
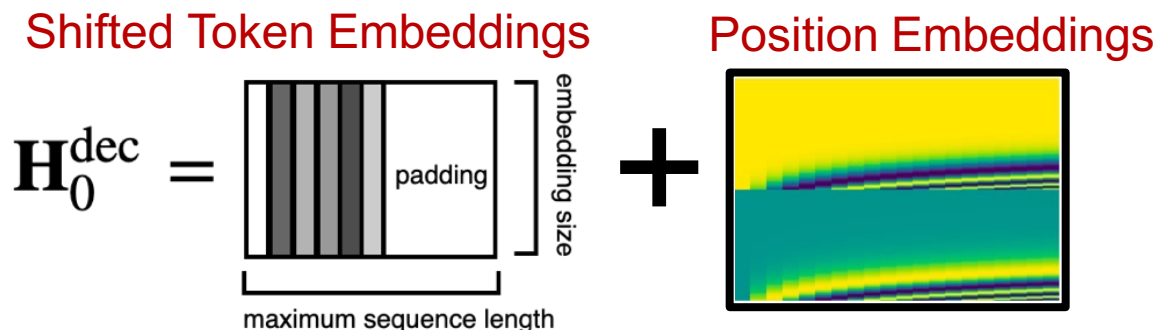


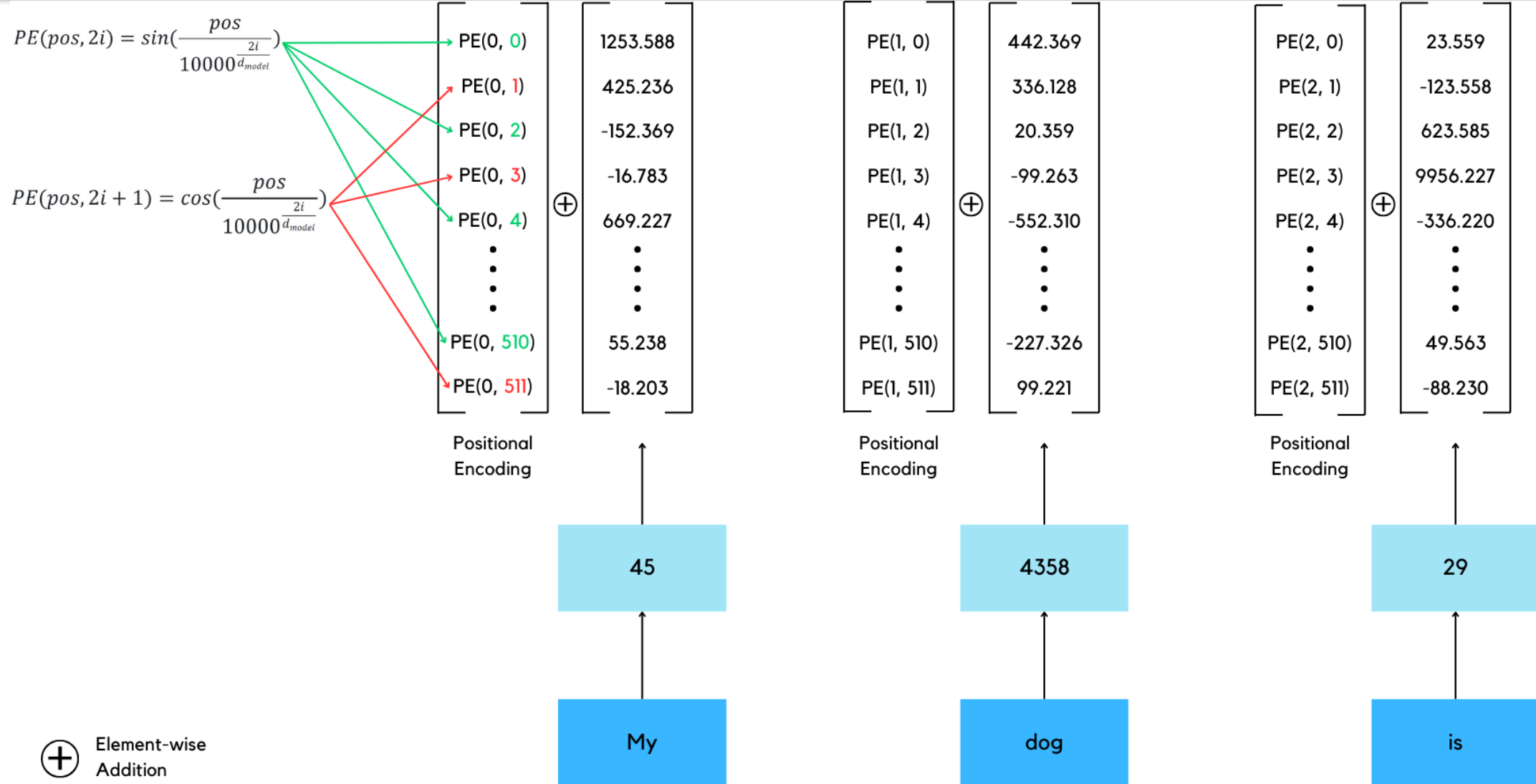
Transformers: “Attention is All You Need”

The input into the encoder looks like:



The input to the decoder looks like:

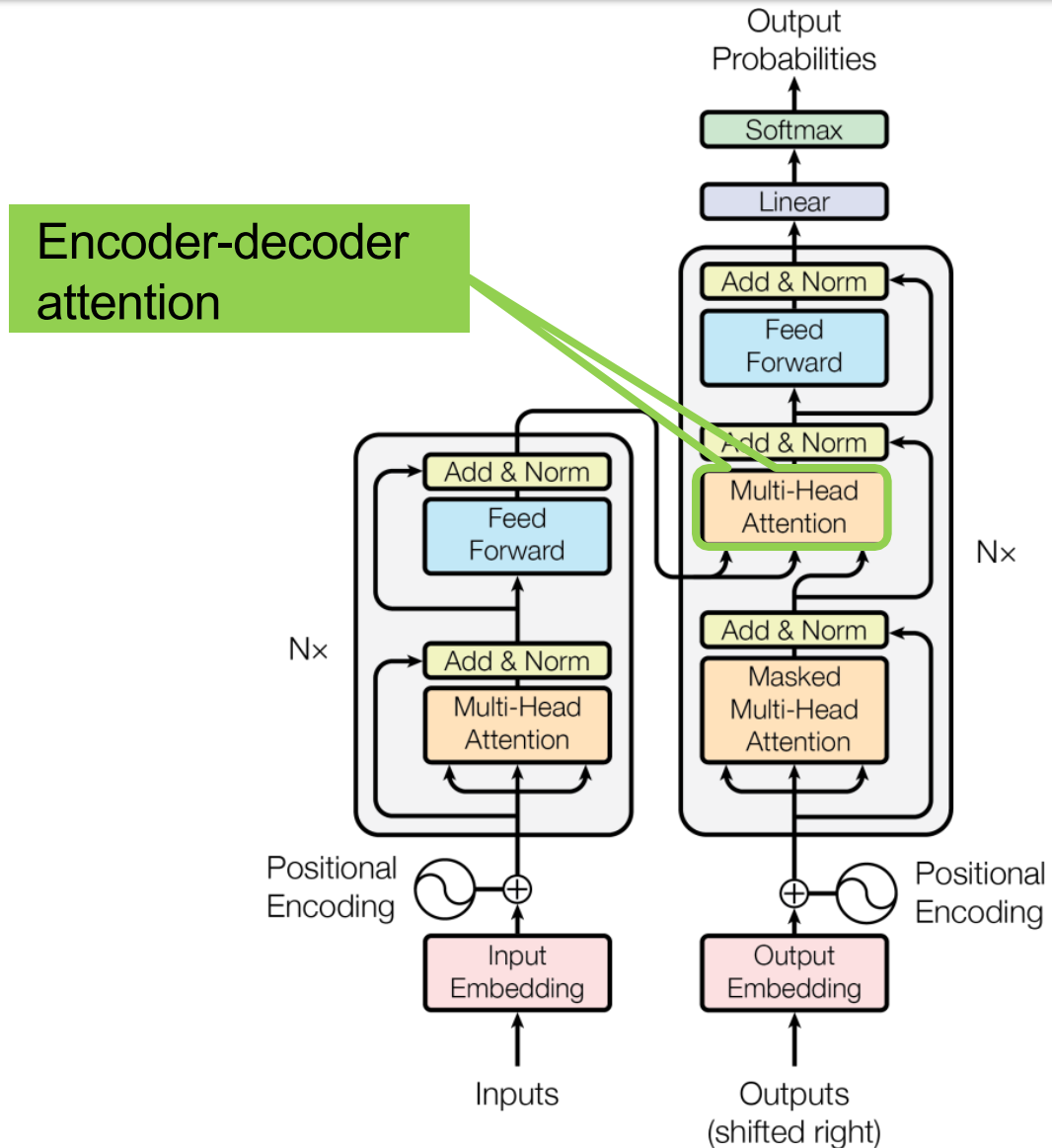




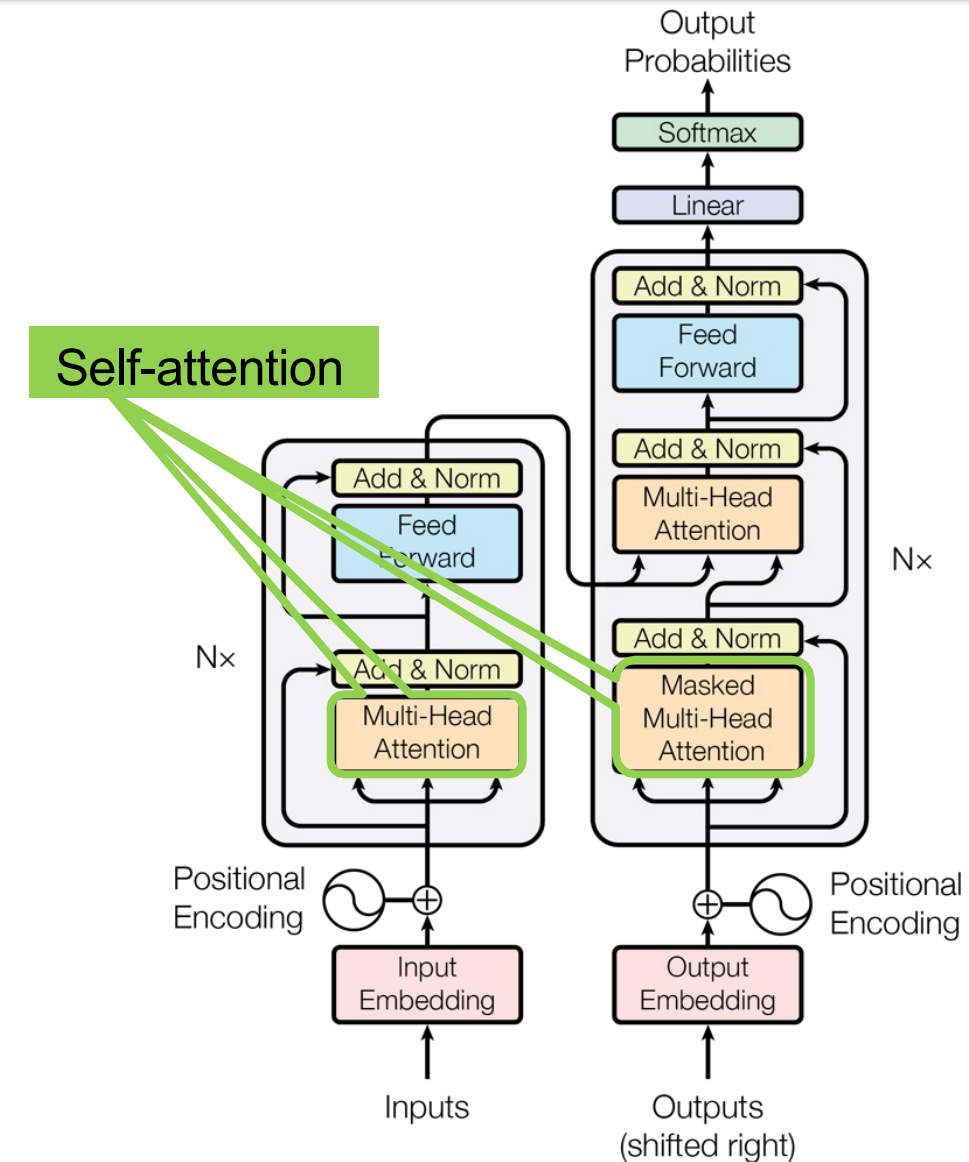
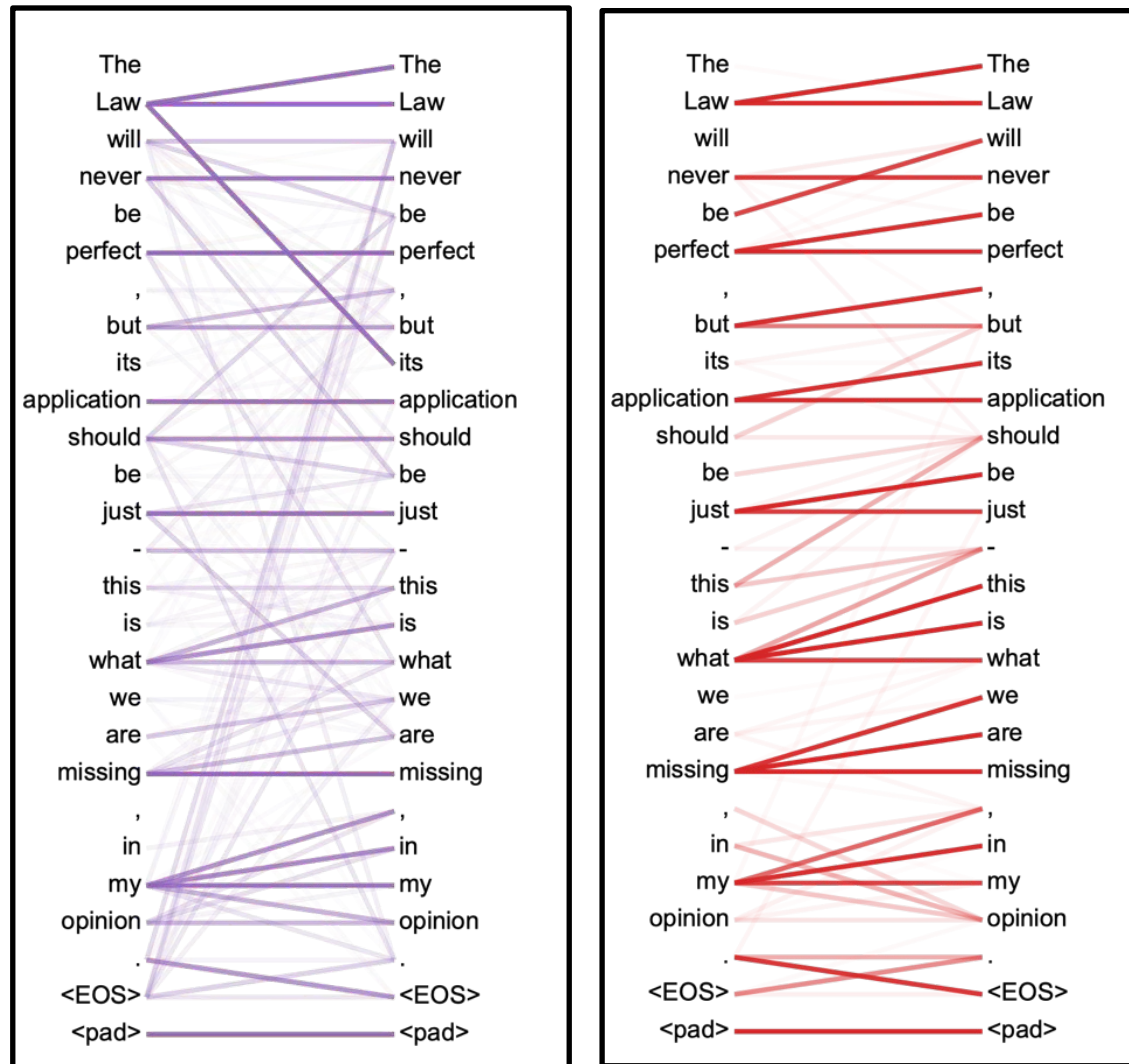
Source: https://github.com/SirawitC/Transformer_from_scratch_pytorch

Transformers: “Attention is All You Need”

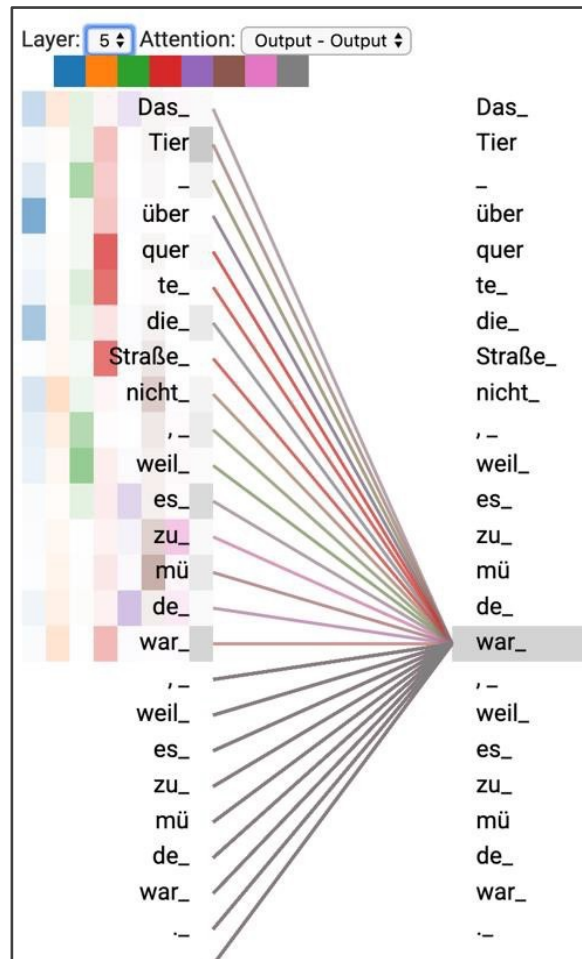
- This is almost exactly the same as what the old recurrent *seq2seq* models had.



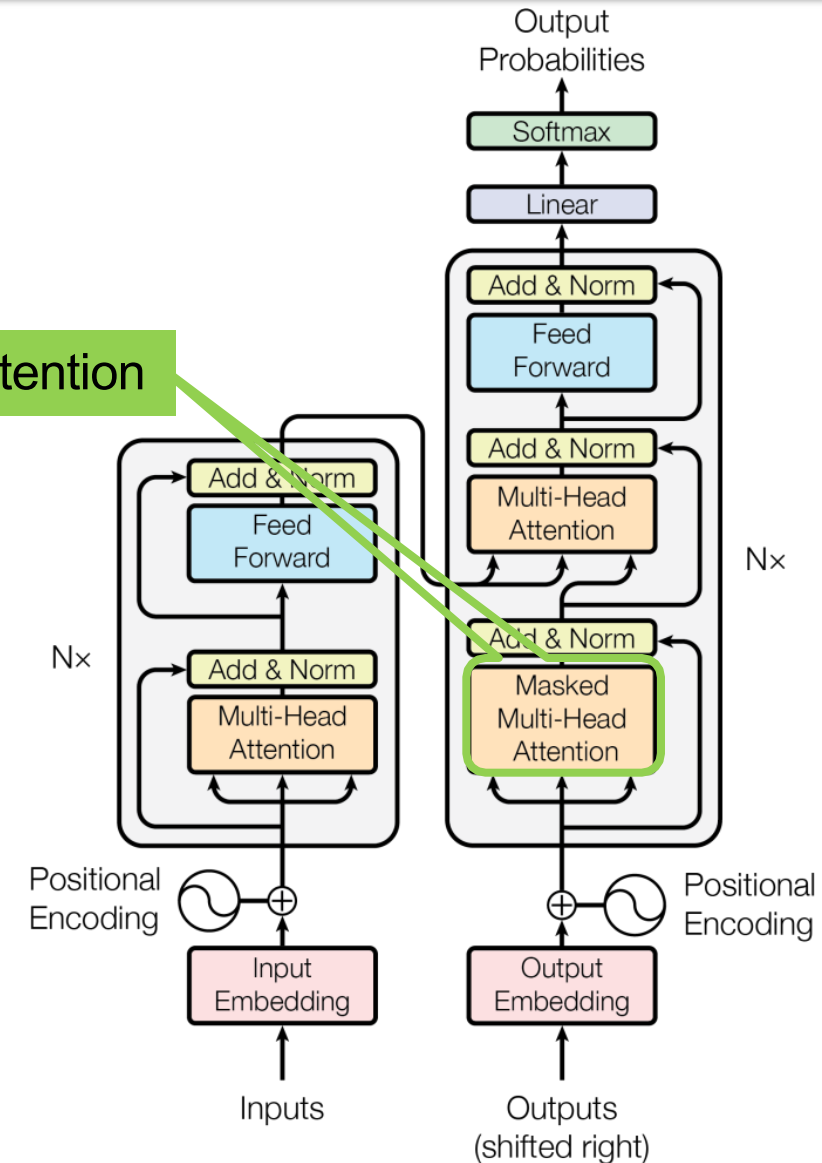
Transformers: “Attention is All You Need”



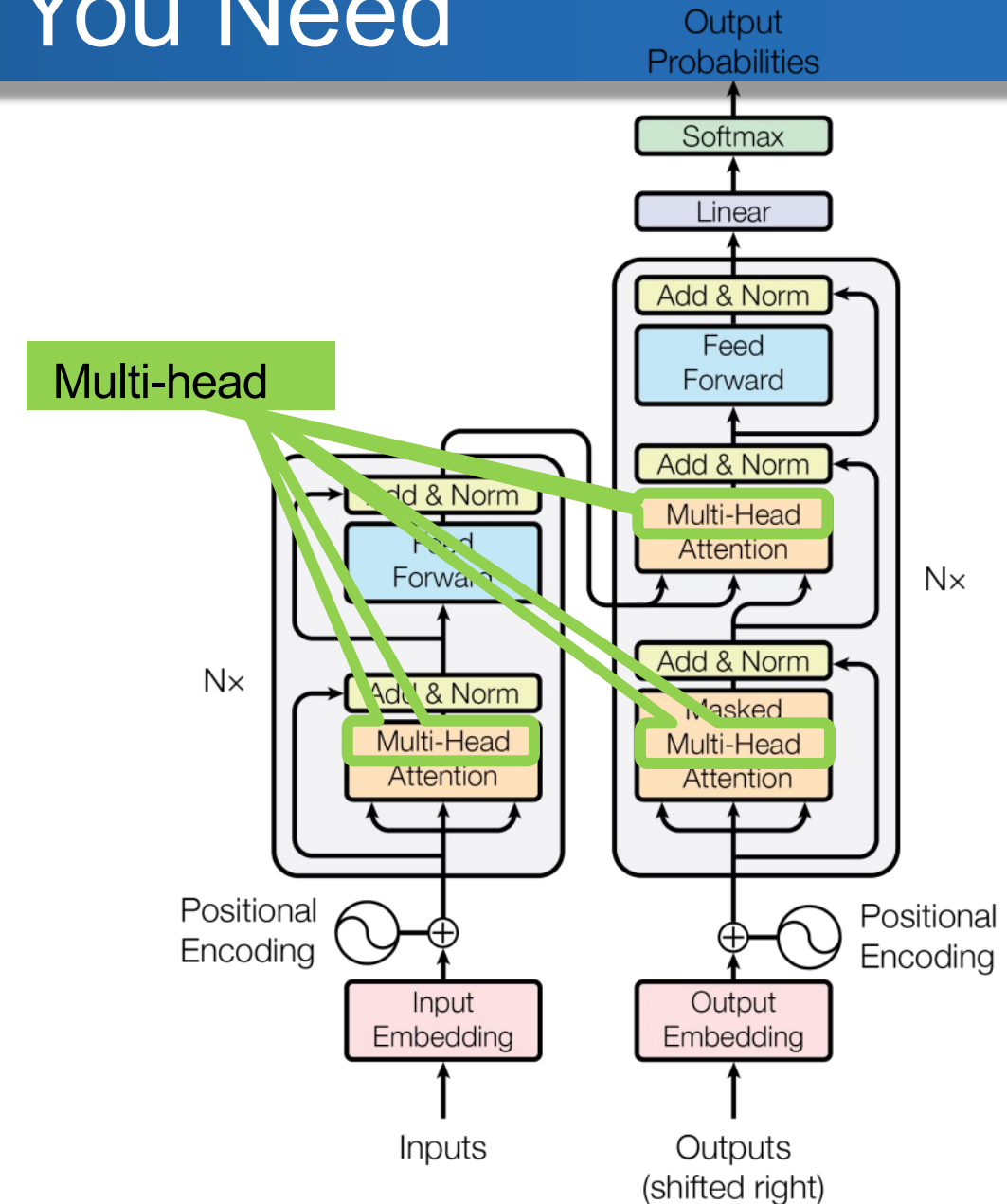
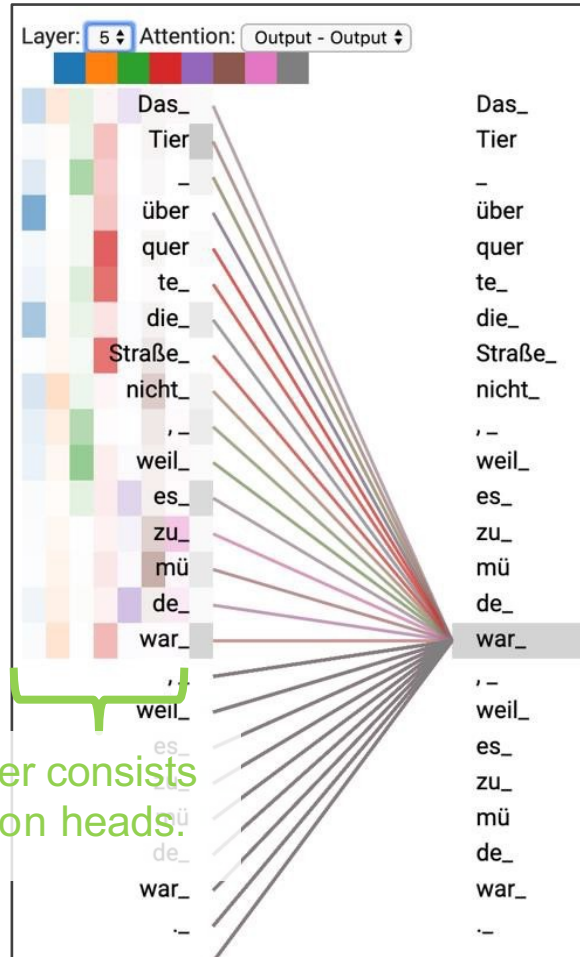
Transformers: “Attention is All You Need”



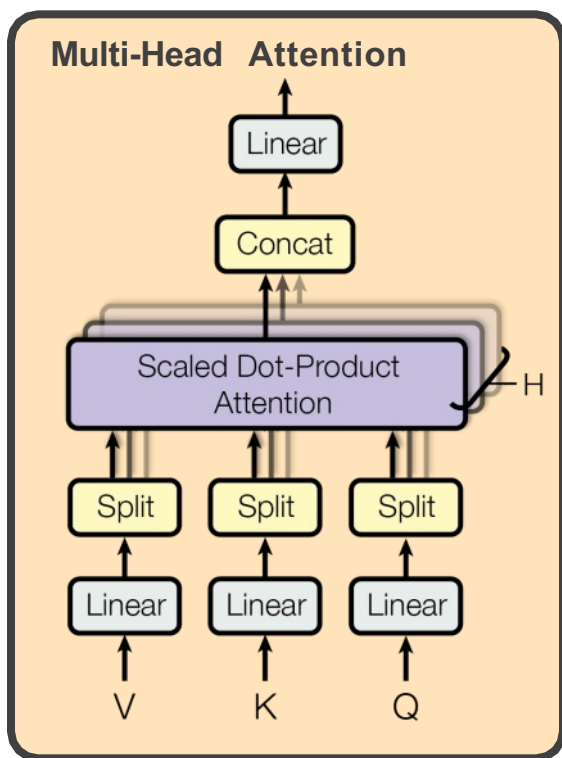
Masked self-attention



Transformers: “Attention is All You Need”



Transformers: “Attention is All You Need”



$$\text{head}_1 = \text{Attention}(QW_1^Q, KW_1^K, VW_1^V)$$

$\vdots$

$$\text{head}_H = \text{Attention}(QW_H^Q, KW_H^K, VW_H^V)$$

$$\text{MultiHeadAtt}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)$$

Inputs and outputs of each layer are the same dimensions:

$$Q \in \mathbb{R}^{T \times d_{\text{model}}}$$

$$K \in \mathbb{R}^{T \times d_{\text{model}}}$$

$$V \in \mathbb{R}^{T \times d_{\text{model}}}$$

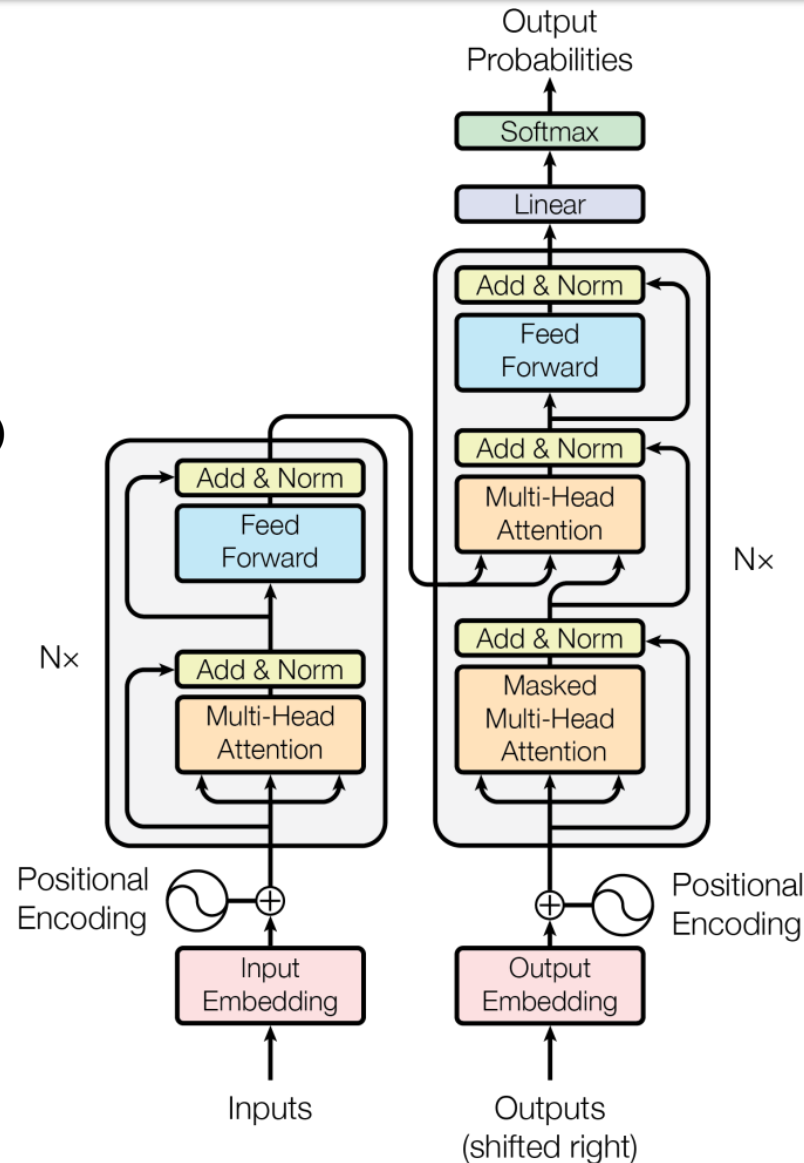
$$\text{MultiHeadAtt}(Q, K, V) \in \mathbb{R}^{T \times d_{\text{model}}}$$

Concrete example:

$$d_{\text{model}} = 512 \text{ and } H = 8$$

$$\text{This means: } W_i^Q \in \mathbb{R}^{512 \times 64}$$

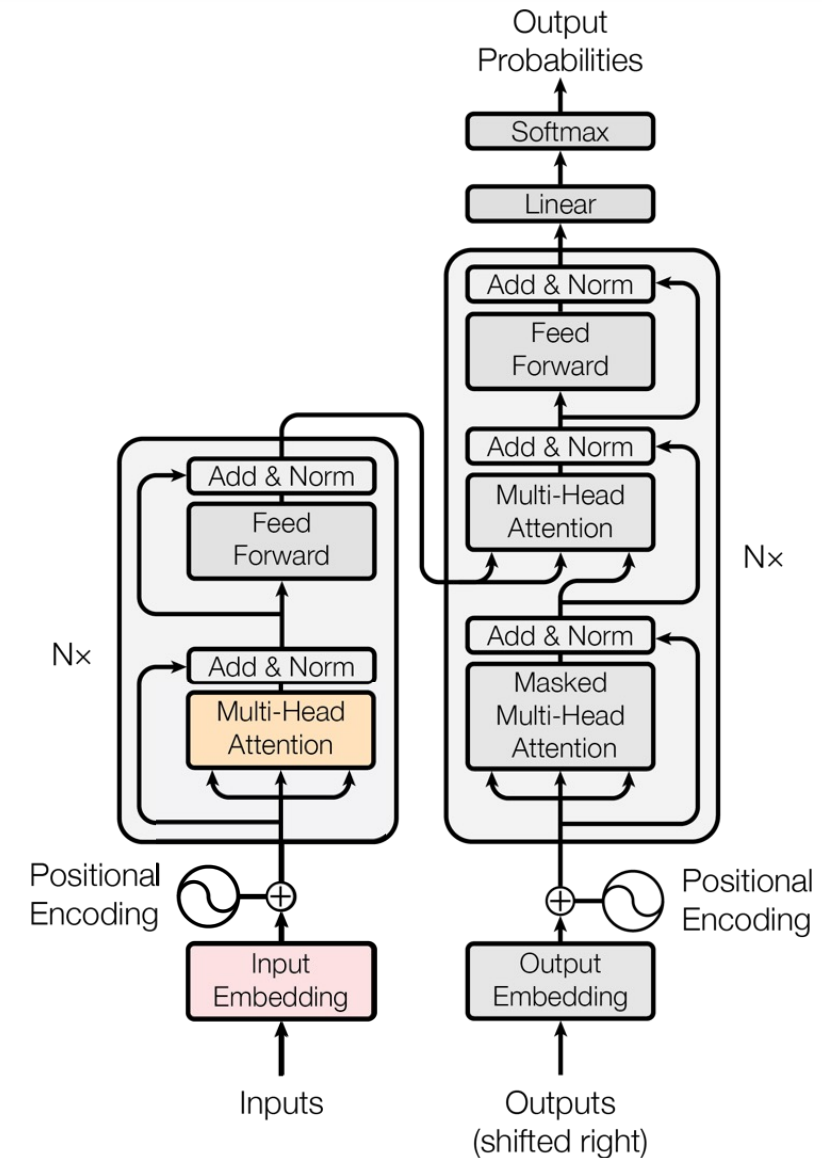
$$W_i^K \in \mathbb{R}^{512 \times 64}, W_i^V \in \mathbb{R}^{512 \times 64}$$



The Encoder Step-by-Step (1)

Multi-Head
Attention

$$= \text{MultiHeadAtt}(\mathbf{H}_i^{\text{enc}}, \mathbf{H}_i^{\text{enc}}, \mathbf{H}_i^{\text{enc}})$$



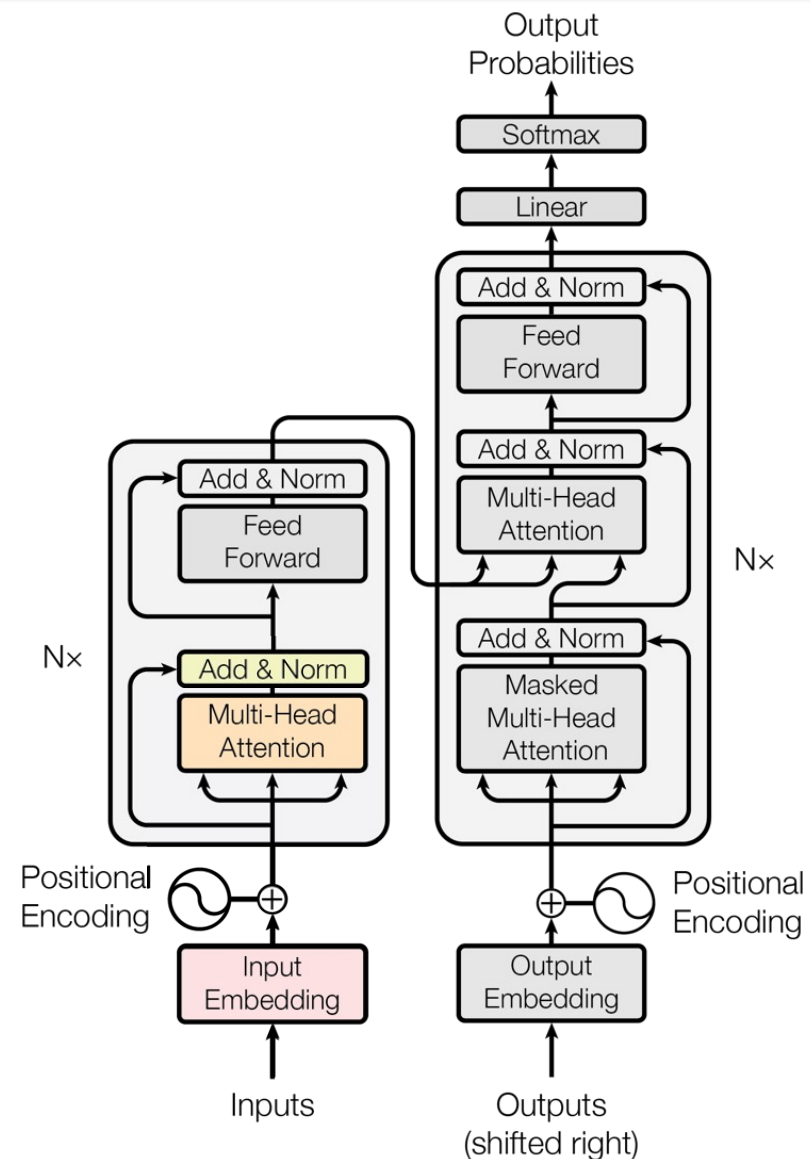
The Encoder Step-by-Step (2)

Multi-Head
Attention

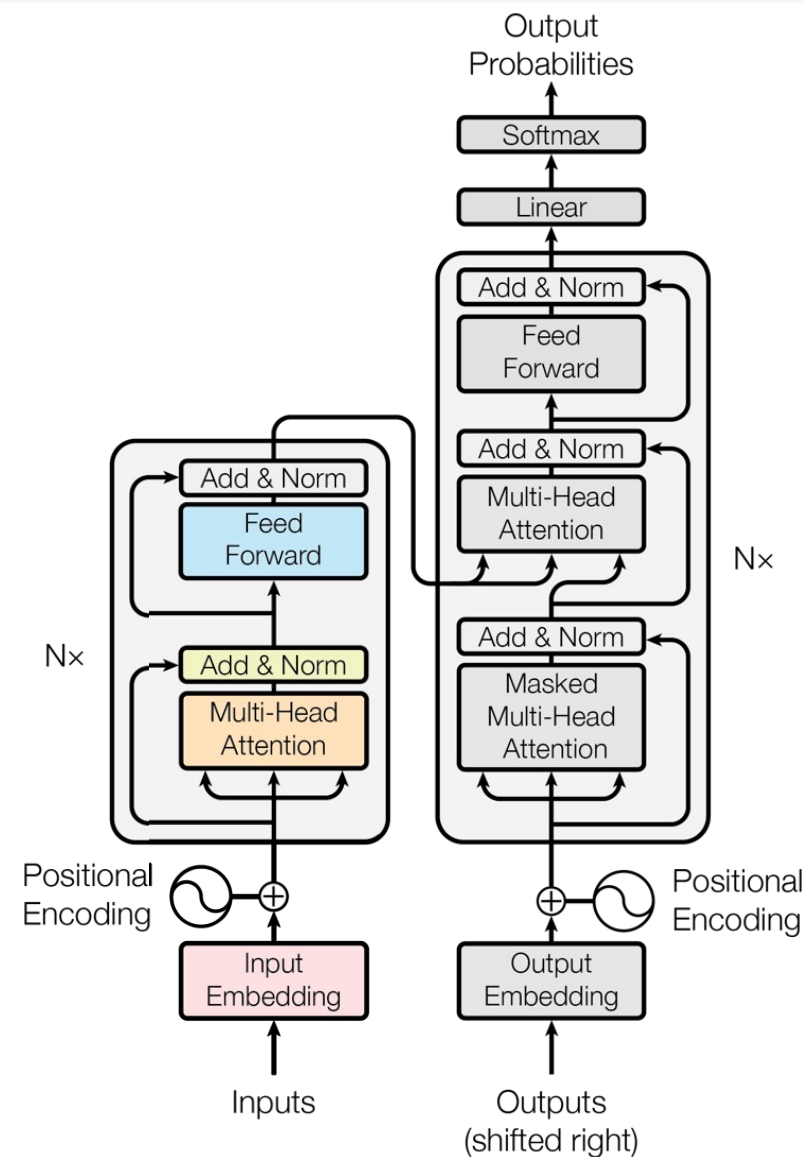
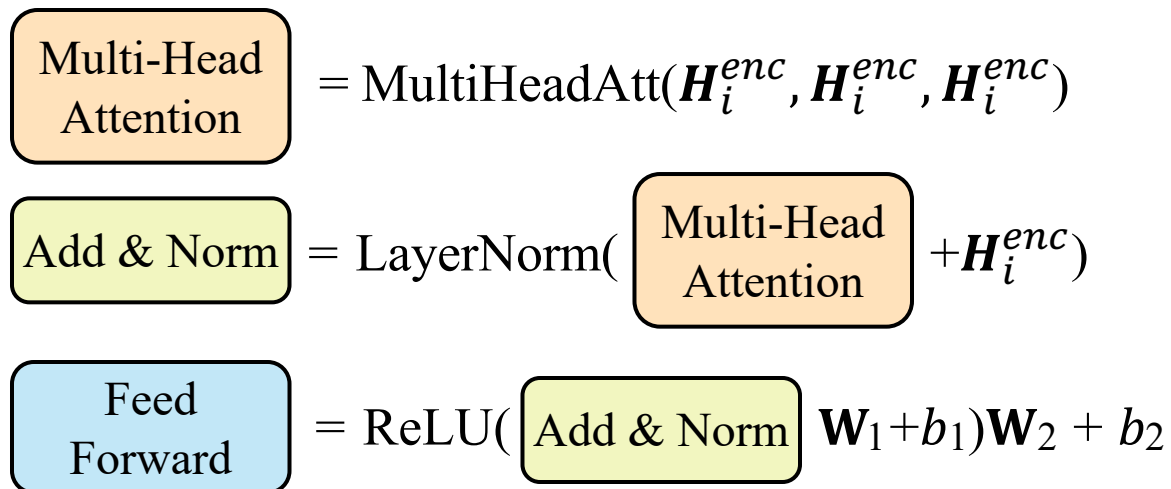
$$= \text{MultiHeadAtt}(\mathbf{H}_i^{\text{enc}}, \mathbf{H}_i^{\text{enc}}, \mathbf{H}_i^{\text{enc}})$$

Add & Norm

$$= \text{LayerNorm}(\text{Multi-Head Attention} + \mathbf{H}_i^{\text{enc}})$$



The Encoder Step-by-Step (3)



The Encoder Step-by-Step (4)

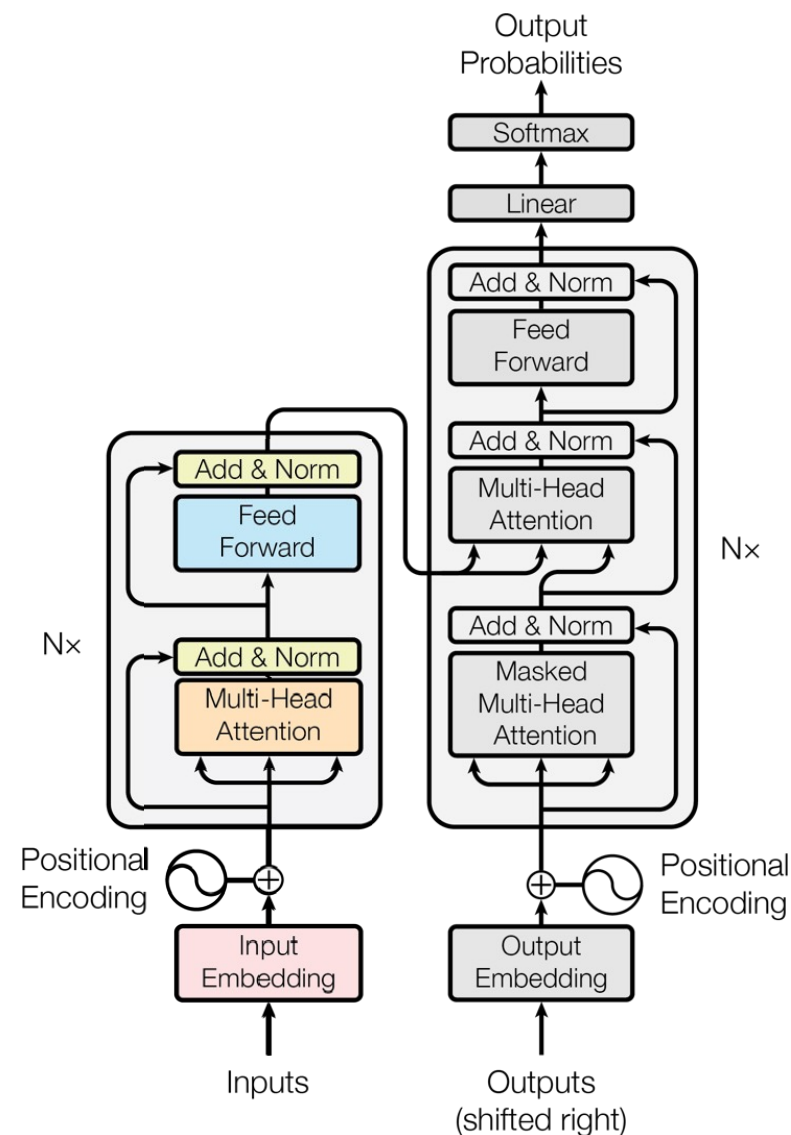
$$\text{Multi-Head Attention} = \text{MultiHeadAtt}(\mathbf{H}_i^{enc}, \mathbf{H}_i^{enc}, \mathbf{H}_i^{enc})$$

$$\text{Add \& Norm} = \text{LayerNorm}(\text{Multi-Head Attention} + \mathbf{H}_i^{enc})$$

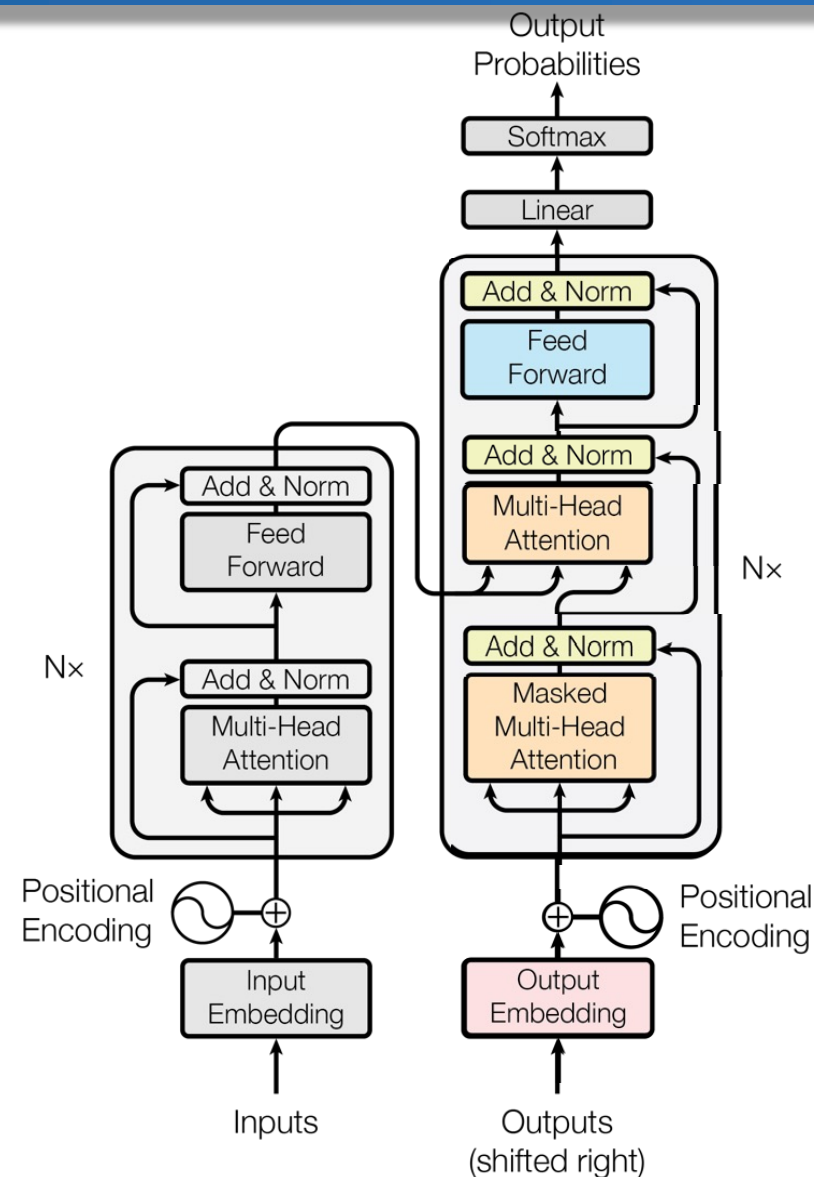
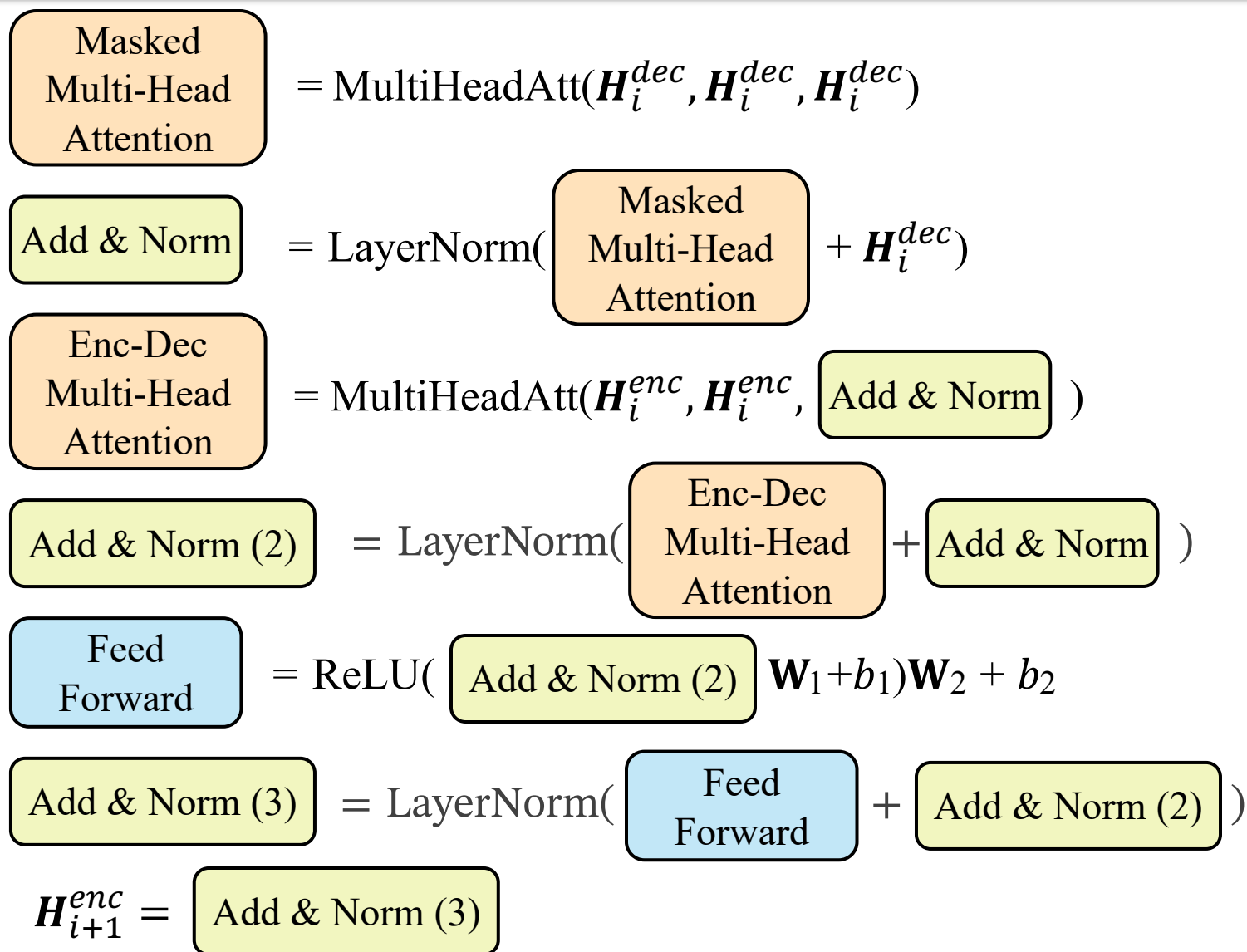
$$\text{Feed Forward} = \text{ReLU}(\text{Add \& Norm} \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$

$$\text{Add \& Norm (2)} = \text{LayerNorm}(\text{Feed Forward} + \text{Add \& Norm})$$

$$\mathbf{H}_{i+1}^{enc} = \text{Add \& Norm (2)}$$

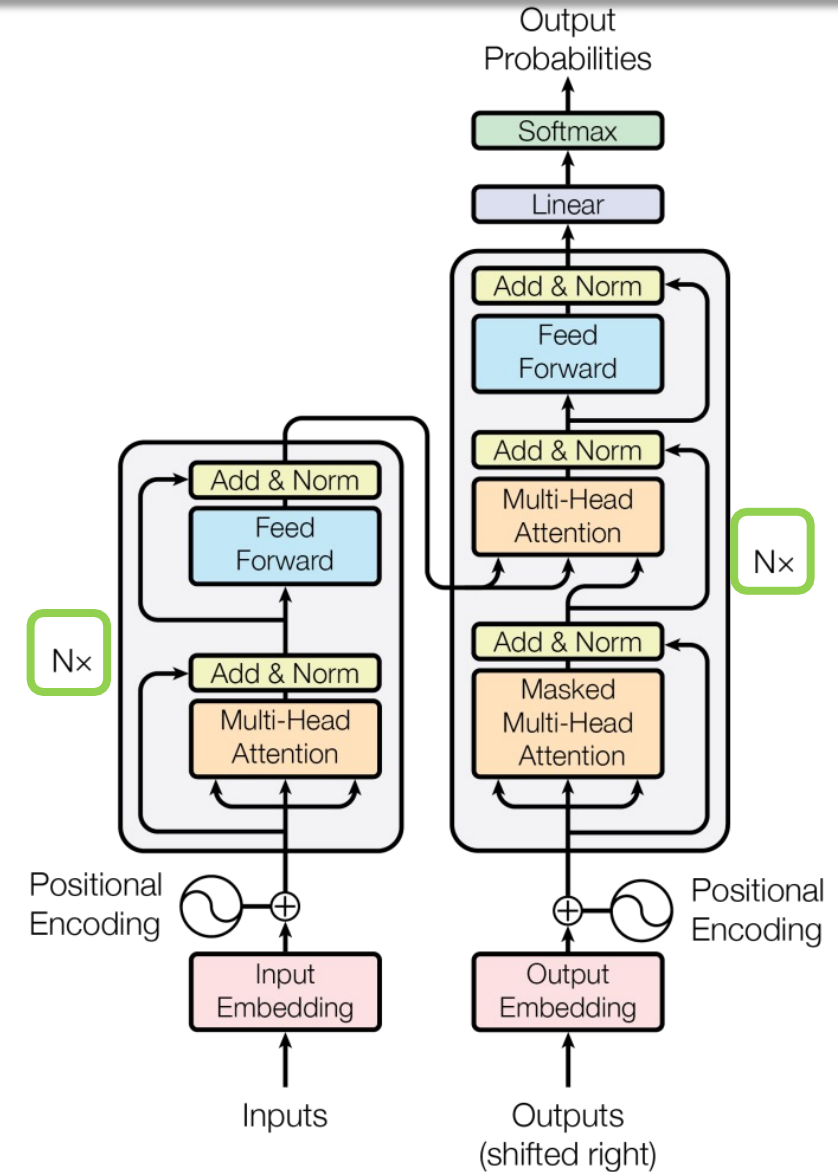


The Decoder Step-by-Step

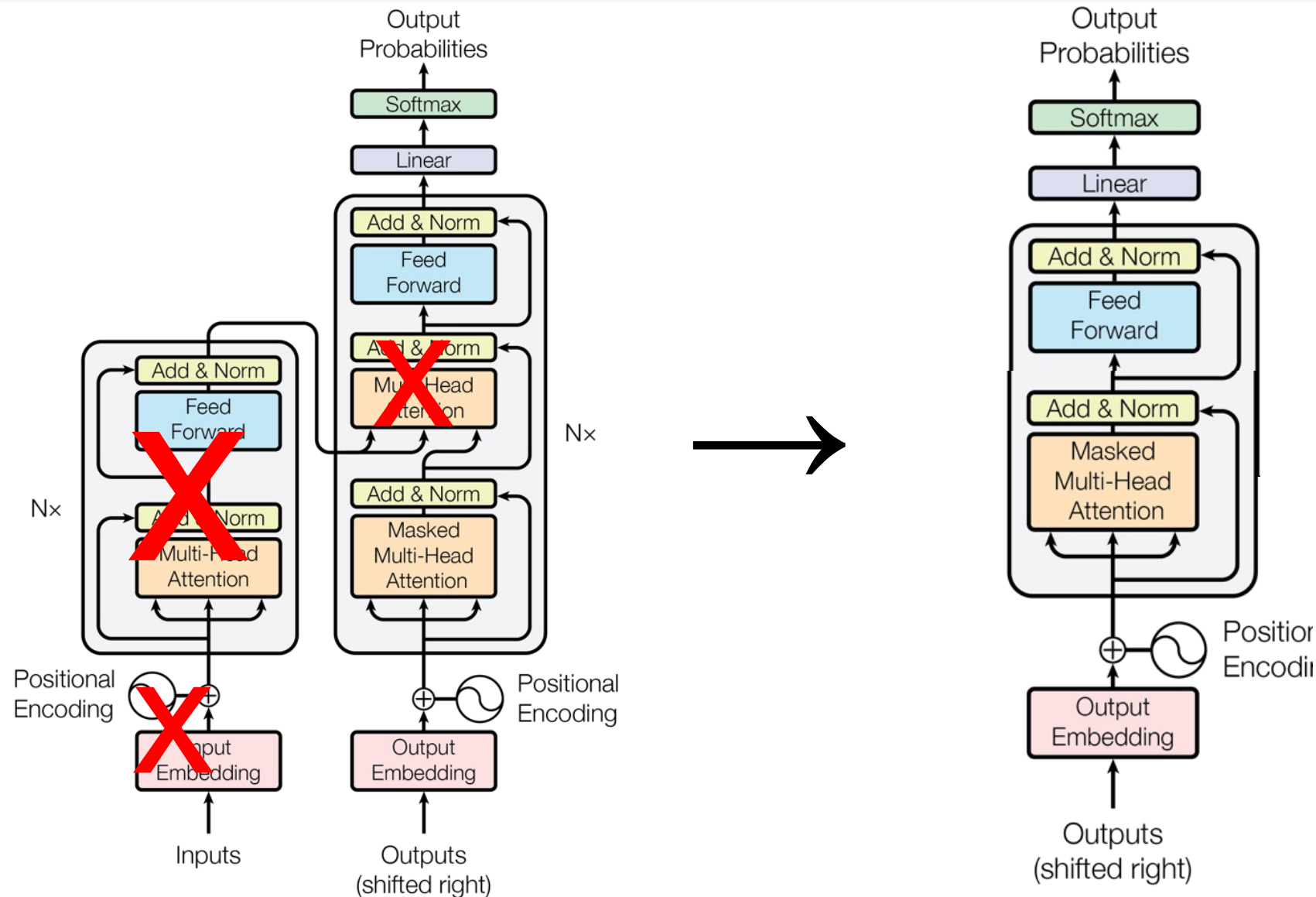


Multiple Layers

- In practice, there are many attention layers.



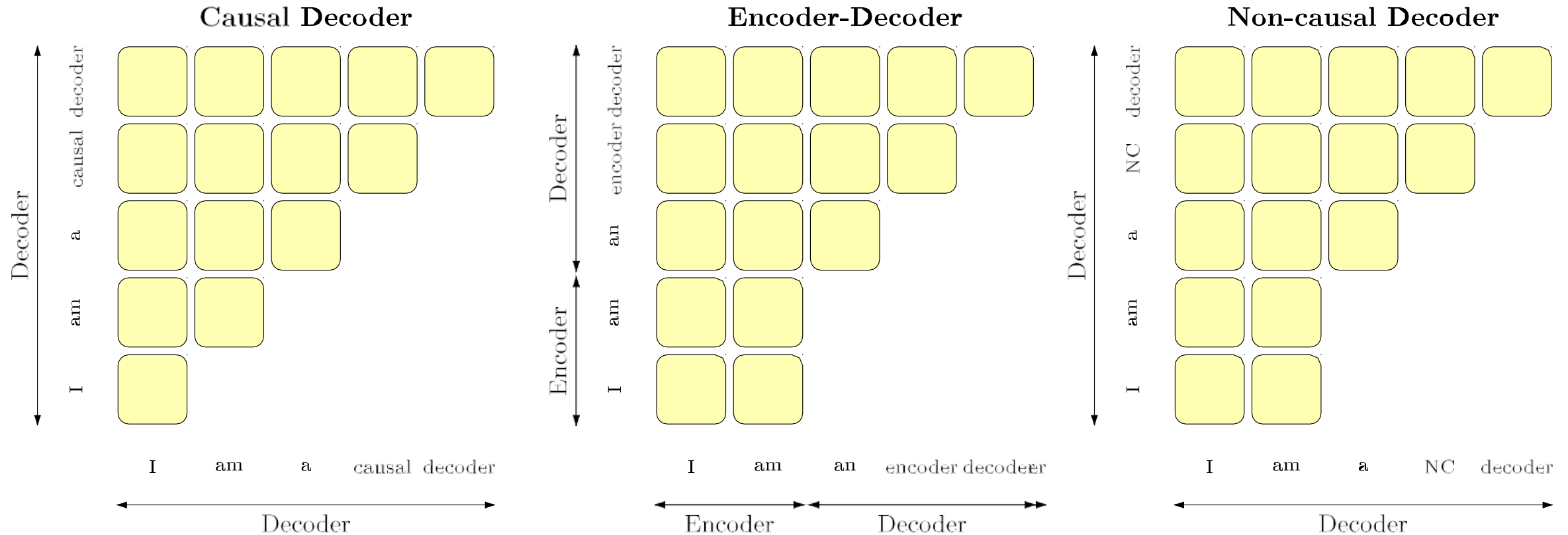
Turning this into a decoder-only architecture



Perplexity

Attention in different Transformer architectures.

Yellow boxes indicate positions that are allowed to attend to each other.



Wang et al. "What Language Model Architecture and Pretraining Objective Work Best for Zero-Shot Generalization?"

Evaluation: How good is our model?

- ❑ Does our language model prefer good sentences to bad ones?
 - Assign higher probability to “real” or “frequently observed” sentences
 - Than “ungrammatical” or “rarely observed” sentences?
- ❑ We train parameters of our model on a **training set**.
- ❑ We test the model’s performance on data we haven’t seen.
 - A **test set** is an unseen dataset that is different from our training set, totally unused.
 - An **evaluation metric** tells us how well our model does on the test set.

Evaluation: How good is our model?

- ❑ The goal isn't to pound out fake sentences!
 - Obviously, generated sentences get “better” as we increase the model order
 - More precisely: using maximum likelihood estimators, higher order is always better likelihood on **training set**, but not **test set**

Example

- Use a bunch of New York Times articles to build a bigram probability table



train

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

evaluate



A good language model
should assign a high
probability to heldout text!

Now I'm going to evaluate the
probability of some *heldout*
data using our bigram table

Training on the test set

- ❑ We can't allow test sentences into the training set
- ❑ We will assign it an artificially high probability when we set it in the test set
- ❑ “Training on the test set”
- ❑ Bad science!

This advice is generally applicable to any downstream task! Do NOT do this in your final projects unless you want to lose a lot of points :)

Perplexity

- The best language model is one that best predicts an unseen test set
 - Gives the highest $P(sentence)$
 - Perplexity is the inverse probability of the test set, normalized by the number of words:

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \end{aligned}$$

Chain rule:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$$

For bigrams:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing perplexity is the same as maximizing probability

Perplexity as branching factor

- Let's suppose a sentence consisting of random digits
- What is the perplexity of this sentence according to a model that assign $P=1/10$ to each digit?

$$\begin{aligned}\text{PP}(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \left(\frac{1}{10}\right)^{-\frac{1}{N}} \\ &= \frac{1}{10}^{-1} \\ &= 10\end{aligned}$$

In practice, we use log probs

$$PP(W) = \exp \left(-\frac{1}{N} \sum_i^N \log P(w_i | w_{<i}) \right)$$

Perplexity is the token-level *negative log-likelihood*

Lower Perplexity = Better Model

- Training 38 million words, test 1.5 million words, Wall Street Journal

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

Shakespeare as corpus

- $N=884,647$ tokens, $V=29,066$
- Shakespeare produced 300,000 bigram types out of $V^2= 844$ million possible bigrams.
 - So 99.96% of the possible bigrams were never seen (have zero entries in the table)

Learning Objectives

What I've told you so far:

- ❑ Language models are trained with the objective of **predicting the next word in a sequence given the previous words** (and possibly some other conditioning signal).
- ❑ We can change the learning objective by changing up these sequences.

What do we want from our learning objective.

- ❑ The goals of *pre-training* (the first stage of training) are to get the language model to:
 - learn the structure of natural language
 - learn humans' understanding of the world (as encoded in the training data)
- ❑ We want a learning objective that facilitates these goals.

Possible objectives for pre-training an encoder-decoder model:

- ❑ Predict a suffix given a prefix.
 - Input: I took my dog, Fido, to the
 - Target: park for his walk.
- ❑ Masked language modeling
 - Input: I took <x> to <y> his walk.
 - Target: <x> my dog, Fido, <y> the park for